

PERSONAL STUDY HANDBOOK ————— Java Edition

DSA Handbook

From Fundamentals to 2-D Arrays

Beginner-Friendly

Interview-Oriented

Deeply Explained

```
int[] arr = {10, 25, 7, 42, 18, 33, 5};
```

10	25	7	42	18	33	5
[0]	[1]	[2]	[3]	[4]	[5]	[6]

Based on **TakeUForward** (Striver), **Love Babbar** & **APNA CLG DSA Series**

• [w3schools.com](https://www.w3schools.com) • [geeksforgeeks.org](https://www.geeksforgeeks.org) • codehelp.in • apnacollege.in

Biraj

ECE Student · Full-Stack Developer

github.com/Biraj2004

Version 1.0

May 2026

Contents

How to Use This Handbook	iv
1 Fundamentals of Programming	1
1.1 What is a Program?	1
1.2 How Java Works	1
1.3 Your First Java Program	2
1.4 Flowcharts and Pseudocode	2
1.4.1 Flowchart Symbols	2
1.4.2 Flowchart: Is a Number Even or Odd?	3
1.4.3 Pseudocode for Even / Odd	3
2 Java Language Basics	4
2.1 Variables and Data Types	4
2.1.1 Java's Eight Primitive Types	4
2.2 Operators	5
2.2.1 Arithmetic Operators	5
2.2.2 Relational and Logical Operators	6
2.3 Input and Output	6
2.3.1 Output Methods — print, println, and printf	7
2.4 Conditional Statements	9
2.4.1 if / else if / else	9
2.4.2 switch-case	10
2.5 Loops	10
2.5.1 for Loop	10
2.5.2 while and do-while Loops	11
2.5.3 break and continue	11
2.6 Methods in Java	12
3 Time and Space Complexity	13
3.1 Why Complexity Matters	13
3.2 Common Complexity Classes	13
3.3 Rules for Analysing Code	14
3.4 Space Complexity	15
4 Basic Mathematics for DSA	16
4.1 Count Digits of a Number	16
4.2 Reverse a Number	17
4.3 Palindrome Number	18
4.4 Factorial of a Number	19
4.5 GCD and LCM	19
4.6 Prime Number Check	20
5 1-D Arrays	22
5.1 What is an Array?	22
5.1.1 Memory Representation	22

5.2	Declaring and Initialising Arrays	23
5.3	Traversing an Array	23
5.4	Sum of Array Elements	24
5.5	Largest and Smallest Element	25
5.6	Second Largest Element	26
5.7	Linear Search	27
5.8	Check if Array is Sorted	28
5.9	Reverse an Array	29
5.10	Count Odd Numbers in Array	30
5.11	Left Rotate Array by One	30
5.12	Left Rotate by K Places	31
5.13	Maximum Consecutive Ones	33
5.14	Move Zeros to End	34
5.15	Remove Duplicates from Sorted Array	35
5.16	Find the Missing Number	36
5.17	Union of Two Sorted Arrays	37
5.18	Intersection of Two Sorted Arrays	38
5.19	Two Sum Problem	39
5.20	Leaders in an Array	41
6	Array Problems — Interview Revision	42
6.1	Complete Problem Reference Table	42
6.2	The Two Pointer Technique	43
6.3	Top Interview Questions	43
6.4	Common Edge Cases to Check	44
7	Sorting Algorithms	45
7.1	Selection Sort	45
7.2	Bubble Sort	47
7.3	Insertion Sort	48
7.4	Merge Sort	49
7.5	Quick Sort	50
7.6	Heap Sort	52
7.7	Radix Sort	54
7.8	Counting Sort	56
7.9	Tim Sort	58
7.10	Sorting Algorithms — When to Use Which	61
8	ArrayList	62
8.1	Creating an ArrayList	62
8.2	Common Operations	63
8.3	Traversal Methods	64
8.4	Size of an ArrayList	65
8.5	Print Reverse of an ArrayList	66
8.6	Swap Two Numbers in an ArrayList	66
8.7	Find Maximum in an ArrayList	67
8.8	Sorting an ArrayList	68
8.9	Multi-Dimensional ArrayList	69
8.10	ArrayList — Interview Corner	71
9	Array Problems — Medium Level	72

9.1	Sort 0s, 1s, and 2s — Dutch National Flag	72
9.2	Majority Element — Boyer-Moore Voting	73
9.3	Kadane's Algorithm — Maximum Subarray Sum	75
9.4	Rearrange Array by Sign	76
9.5	Pascal's Triangle	77
9.6	3-Sum Problem	79
9.7	Equilibrium Index	81
9.8	Missing Elements From an Array With Duplicates	82
9.9	4-Sum Problem	85
9.10	Next Permutation	86
10	2-D Arrays (Matrices)	89
10.1	Memory Representation	89
10.2	Declaring and Initialising a 2-D Array	90
10.3	Print Diagonal Elements	91
10.4	Sum of Diagonal Elements	91
10.5	Search Element in a Matrix	92
10.5.1	Brute Force — Check All Elements	92
10.5.2	Optimised — Staircase Search (Sorted Matrix)	93
10.6	Check if Matrix is Symmetric	94
10.7	Matrix Transpose	96
10.8	Rotate Matrix 90° Clockwise	97
10.9	Rotate Matrix 90° Anticlockwise	98
10.10	Spiral Matrix — Print in Spiral Order	100
10.11	2-D Arrays — Quick Revision	103
11	Array Problems — Hard Level	104
11.1	Merge Two Sorted Arrays Without Extra Space	104
11.2	Maximum Product Subarray	106
11.3	Find the Repeating and Missing Number	107
11.4	Majority Element II	109
11.5	Count Inversions	111
11.6	Reverse Pairs	113
11.7	Trapping Rainwater	115
11.7.1	Brute Force — $O(n^2)$	115
11.7.2	Prefix Array Approach — $O(n)$ time $O(n)$ space	116
11.7.3	Two Pointer Approach — $O(n)$ time $O(1)$ space (Optimal)	117
11.8	Best Time to Buy and Sell Stock	119

How to Use This Handbook

This handbook is your structured companion for learning Data Structures & Algorithms in Java — from the very first concept to mastering 2-D Arrays. It is written to be **shallow-to-medium depth**: clear enough to understand quickly, deep enough to perform in interviews.

Structure of Each Topic

1. **Concept** — What it is, in plain English
2. **Java Code** — Clean implementation (no step-jumps; every line explained)
3. **Dry Run** — Step-by-step table trace on a small example
4. **Visual Diagram** — Memory or execution diagram where helpful
5. **Complexity** — Time and Space Big-O analysis
6. **Interview Corner** — What interviewers ask, edge cases, tricks

Box Colour Guide

Blue box — Core definition / concept

Orange box (Dry Run) — Step-by-step trace table

Indigo box (Complexity) — Big-O analysis

Red box (Interview Corner) — Interview questions & edge cases

Green box (Tip) — Shortcuts, tricks, common mistakes

Fundamentals of Programming

1.1 What is a Program?

A **program** is a set of instructions given to a computer to solve a problem. You write those instructions in a **programming language** (like Java), and the computer executes them one by one.

Essential Terms

- **Source Code** — The human-readable instructions you type
- **Compiler** — Translates source code into machine-understandable code
- **JVM (Java Virtual Machine)** — Runs compiled Java bytecode on any operating system
- **Algorithm** — A finite, step-by-step procedure that solves a problem
- **Data Structure** — A way to organise, store, and access data efficiently

1.2 How Java Works

Java follows a two-step process: compile once, run anywhere.



1. You write code in a `.java` file
2. `javac` compiles it to a `.class` file (bytecode)
3. The JVM reads bytecode and executes it on your machine
4. The result appears as output (console, file, etc.)

Key advantage of JVM: The same `.class` file runs on Windows, Linux, or macOS without recompilation — “Write Once, Run Anywhere.”

1.3 Your First Java Program

```
1 // A comment: ignored by compiler, just for you to read
2
3 public class HelloWorld {           // every Java file needs a class
4                                     // class name MUST match filename
5
6     // main() is the entry point - JVM starts here
7     public static void main(String[] args) {
8
9         // println prints text then moves to the next line
10        System.out.println("Hello, World!");
11
12    } // end of main
13 } // end of class
```

Listing 1.1: Hello World in Java — Every line explained

Dry Run: Hello World

1. JVM locates `main()` and enters it
2. Executes `System.out.println("Hello, World!")` — sends text to console
3. Prints: `Hello, World!`
4. `main()` reaches its closing brace `}` and exits
5. Program terminates

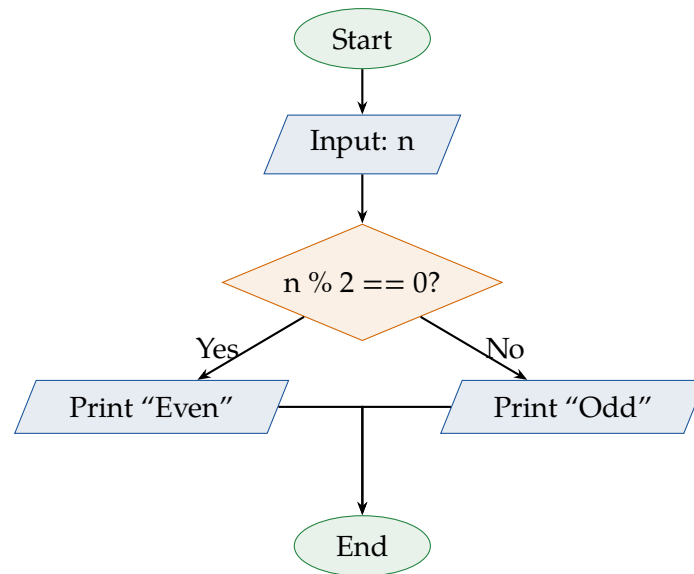
1.4 Flowcharts and Pseudocode

Before writing code, professionals plan their logic using flowcharts or pseudocode. This catches errors early and makes coding faster.

1.4.1 Flowchart Symbols

Shape	Name	Represents
Oval (rounded box)	Terminator	Start or End of program
Rectangle	Process	A computation or action
Diamond	Decision	A yes/no branch (if/else)
Parallelogram	Input/Output	Reading input or printing output
Arrow	Flow line	Direction of execution

1.4.2 Flowchart: Is a Number Even or Odd?



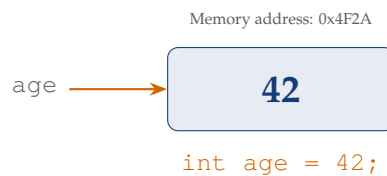
1.4.3 Pseudocode for Even / Odd

```
BEGIN
  INPUT n
  IF n % 2 == 0 THEN
    PRINT "Even"
  ELSE
    PRINT "Odd"
  END IF
END
```


Java Language Basics

2.1 Variables and Data Types

A **variable** is a named slot in memory. You give it a name, declare its type, and assign a value.



2.1.1 Java's Eight Primitive Types

Type	Size	Value Range (approx)	Default	Example
byte	1 byte	−128 to 127	0	byte b = 100;
short	2 bytes	−32,768 to 32,767	0	short s = 1000;
int	4 bytes	≈ −2.1B to +2.1B	0	int n = 42;
long	8 bytes	≈ ±9.2 × 10 ¹⁸	0L	long l = 99L;
float	4 bytes	≈ ±3.4 × 10 ³⁸	0.0f	float f = 3.14f;
double	8 bytes	≈ ±1.7 × 10 ³⁰⁸	0.0	double d = 3.14;
char	2 bytes	0 to 65,535 (Unicode)	'\u0000'	char c = 'A';
boolean	1 bit	true / false	false	boolean ok = true;

```

1 public class DataTypesDemo {
2     public static void main(String[] args) {
3
4         // Integer family
5         int age      = 21;
6         long bigNum   = 7_800_000_000L; // L suffix required for long
7         short s       = 500;

```

```
8      byte b          = 127;           // max value for byte
9
10     // Decimal family
11     double pi        = 3.14159265;    // prefer double for accuracy
12     float temp       = 36.6f;         // f suffix required for float
13
14     // Character and boolean
15     char grade       = 'A';           // single quotes for char
16     boolean pass     = true;
17
18     // String -- NOT a primitive; it is a class
19     String name      = "Riya";        // double quotes for String
20
21     System.out.println("Name   : " + name);
22     System.out.println("Age    : " + age);
23     System.out.println("Grade  : " + grade);
24     System.out.println("PI     : " + pi);
25     System.out.println("Pass   : " + pass);
26 }
27 }
```

Listing 2.1: Variables and Data Types in Action

int vs long vs double

Use `int` for whole numbers that fit in 2 billion. Use `long` for very large counts (populations, timestamps). Use `double` for decimals in DSA — it is the most precise floating-point type in Java.

2.2 Operators

2.2.1 Arithmetic Operators

Op	Meaning	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3 (decimal dropped!)
%	Modulus/Remainder	10 % 3	1

Integer Division Trap

`int / int` always drops the decimal part. To get the true decimal result, cast one operand: `(double) 10 / 3` gives `3.3333...`

2.2.2 Relational and Logical Operators

Category	Operator	Meaning	Example → Result
Relational	<code>==</code>	Equal to	<code>5 == 5 → true</code>
	<code>!=</code>	Not equal	<code>5 != 3 → true</code>
	<code>></code>	Greater than	<code>7 > 3 → true</code>
	<code><</code>	Less than	<code>2 < 9 → true</code>
	<code>>=</code>	Greater or equal	<code>5 >= 5 → true</code>
	<code><=</code>	Less or equal	<code>3 <= 4 → true</code>
Logical	<code>&&</code>	AND — both true	<code>T && F → false</code>
	<code> </code>	OR — at least one	<code>T F → true</code>
	<code>!</code>	NOT — negate	<code>!true → false</code>

2.3 Input and Output

```

1  import java.util.Scanner;    // import this class before using it
2
3  public class InputDemo {
4      public static void main(String[] args) {
5
6          // Create a Scanner connected to the keyboard (System.in)
7          Scanner sc = new Scanner(System.in);
8
9          System.out.print("Enter your name: ");
10         String name = sc.nextLine();    // reads the whole line
11
12         System.out.print("Enter your age: ");
13         int age = sc.nextInt();        // reads one integer
14
15         System.out.print("Enter your GPA: ");
16         double gpa = sc.nextDouble();  // reads a decimal number
17
18         // Output
19         System.out.println("Name : " + name);
20         System.out.println("Age : " + age);
21         System.out.printf("GPA : %.2f\n", gpa); // 2 decimal places
22     }
}

```

```

23     sc.close();    // always close the Scanner
24 }
25 }

```

Listing 2.2: Reading User Input with Scanner

Scanner Method	What it Reads
<code>sc.nextInt()</code>	One integer
<code>sc.nextLong()</code>	One long value
<code>sc.nextDouble()</code>	One decimal (double)
<code>sc.nextFloat()</code>	One decimal (float)
<code>sc.next()</code>	One word (stops at whitespace)
<code>sc.nextLine()</code>	A complete line of text
<code>sc.nextBoolean()</code>	true or false

nextInt() + nextLine() Pitfall

After `sc.nextInt()`, the newline character `\n` stays in the buffer. If you call `sc.nextLine()` immediately after, it picks up that leftover newline. Fix: add an extra `sc.nextLine()` to consume it before reading the string.

2.3.1 Output Methods — print, println, and printf

Java has three ways to print output. Knowing when to use which is essential.

Method	What it does	Example output
<code>System.out.println()</code>	Prints then moves to next line automatically	Hello (cursor → next line)
<code>System.out.print()</code>	Prints with no newline — cursor stays on same line	Hello (cursor stays)
<code>System.out.printf()</code>	Prints using a format string with placeholders	GPA : 8.46

```

1 public class OutputMethods {
2     public static void main(String[] args) {
3
4         // println: adds a newline at the end automatically
5         System.out.println("Hello");    // prints "Hello" then goes to next line
6         System.out.println("World");    // prints "World" on a new line
7         // Output:

```

```

8      // Hello
9      // World

10
11     // print: NO newline added; cursor stays on the same line
12     System.out.print("Hello ");
13     System.out.print("World");
14     System.out.println();    // manual newline to end the line
15     // Output:
16     // Hello World

17
18     // printf: uses a FORMAT STRING with placeholders
19     //  %s  --> placeholder for a String
20     //  %d  --> placeholder for an integer (int)
21     //  %f  --> placeholder for a decimal (double / float)
22     //  %.2f --> decimal with exactly 2 digits after the decimal point
23     //  %n  --> newline (platform-independent, prefer over \n in printf)
24
25     String name = "Riya";
26     int age = 20;
27     double gpa = 8.4567;
28
29     System.out.printf("%s is %d years old%n", name, age);
30     // Output: Riya is 20 years old
31
32     System.out.printf("GPA : %.2f%n", gpa);
33     // Output: GPA : 8.46    (rounded to 2 decimal places)
34
35     // Multiple placeholders: pass values after the format string, in order
36     System.out.printf("Name: %s | Age: %d | GPA: %.1f%n", name, age, gpa);
37     // Output: Name: Riya | Age: 20 | GPA: 8.5
38 }
39 }

```

Listing 2.3: print vs println vs printf — all three compared

How printf Format Strings Work

`printf()` uses a **format string** first, then the actual values separated by commas.

Placeholder	Meaning
%s	String
%d	Integer (int, long)
%f	Floating-point (double, float) — default 6 decimal places
%.2f	Float with exactly 2 decimal places (.2 controls precision)
%n	Newline (use instead of \n inside printf)
%b	Boolean

Example: `System.out.printf("GPA : %.2f%n", gpa);`

The placeholder `%.2f` is *replaced* by the value of `gpa`, formatted to 2 decimal places.

`double gpa = 8.4567; printf → GPA : 8.46`

When to use which?

println — most common; use for simple messages, debug output, and when each print goes on its own line.

print — use when you want multiple values on the *same line* (e.g., printing array elements separated by spaces).

printf — use when you need *formatted* output: fixed decimal places, aligned columns, or embedding variables cleanly inside a string.

2.4 Conditional Statements

2.4.1 if / else if / else

```
1 public class GradeCalc {
2     public static void main(String[] args) {
3         int marks = 75;
4
5         if (marks >= 90) {
6             System.out.println("Grade: A");
7         } else if (marks >= 75) {
8             System.out.println("Grade: B");    // <-- this runs
9         } else if (marks >= 60) {
10            System.out.println("Grade: C");
11        } else if (marks >= 40) {
12            System.out.println("Grade: D");
13        } else {
14            System.out.println("Grade: F");
15        }
16    }
17 }
```

Listing 2.4: Grade Calculator using if-else chain

Dry Run: marks = 75

1. Check `75 >= 90`? No → move to next condition
2. Check `75 >= 75`? **Yes** → execute this block
3. Print `Grade: B`
4. All remaining `else if` and `else` blocks are skipped

Output: `Grade: B`

2.4.2 switch-case

Use `switch` when comparing one variable against many fixed values — it is often cleaner than a long `else if` chain.

```
1 public class DayName {
2     public static void main(String[] args) {
3         int day = 3;
4
5         switch (day) {
6             case 1: System.out.println("Monday"); break;
7             case 2: System.out.println("Tuesday"); break;
8             case 3: System.out.println("Wednesday"); break; // runs
9             case 4: System.out.println("Thursday"); break;
10            case 5: System.out.println("Friday"); break;
11            default: System.out.println("Weekend");
12        }
13    }
14 }
```

Listing 2.5: Day Name using switch-case

Always Write break!

Without `break`, Java *falls through* into the next case and executes it too. This is one of the most common beginner bugs.

2.5 Loops

2.5.1 for Loop

Use the `for` loop when you know **exactly** how many times to repeat.

```
1 public class ForLoopDemo {
2     public static void main(String[] args) {
3
4         //   init   condition   update
5         for (int i = 1; i <= 5; i++) {
6             System.out.println("i = " + i);
7         }
8     }
9 }
```

Listing 2.6: for Loop — Print 1 to 5

Dry Run: for loop i = 1 to 5

Iteration	i	i ≤ 5?	Output
1	1	true	i = 1
2	2	true	i = 2
3	3	true	i = 3
4	4	true	i = 4
5	5	true	i = 5
6	6	false	Loop ends

2.5.2 while and do-while Loops

```

1 public class WhileDoWhile {
2     public static void main(String[] args) {
3
4         // while: checks condition BEFORE entering the body
5         int i = 1;
6         while (i <= 5) {
7             System.out.print(i + " ");
8             i++;           // MUST update -- else infinite loop!
9         }
10        System.out.println(); // Output: 1 2 3 4 5
11
12        // do-while: runs the body FIRST, then checks condition
13        // Body executes at least once even if condition is false
14        int j = 10;
15        do {
16            System.out.println("Ran at least once: j = " + j);
17            j++;
18        } while (j <= 5); // false immediately, but body ran once
19    }
20 }
```

Listing 2.7: while vs do-while — key difference shown

2.5.3 break and continue

```

1 public class BreakContinue {
2     public static void main(String[] args) {
3
4         // break: immediately exits the loop
5         System.out.print("break at 6: ");
6         for (int i = 1; i <= 10; i++) {
7             if (i == 6) break;           // stop here
8             System.out.print(i + " ");
9         }
10    }
```



```
10      // Output: 1 2 3 4 5
11
12      System.out.println();
13
14      // continue: skip the current iteration and move to next
15      System.out.print("skip evens: ");
16      for (int i = 1; i <= 10; i++) {
17          if (i % 2 == 0) continue; // skip even numbers
18          System.out.print(i + " ");
19      }
20      // Output: 1 3 5 7 9
21  }
22 }
```

Listing 2.8: break exits the loop; continue skips the current iteration

Which Loop to Use?

for loop — Number of iterations known in advance (most DSA problems).

while loop — Iterate until a condition changes (count unknown).

do-while — Must run at least once (e.g., menu-driven programs).

2.6 Methods in Java

A **method** is a reusable block of code with a name. It avoids repeating the same code.

```
1  public class MethodsDemo {
2
3      // Method definition: returnType methodName(parameters)
4      public static int add(int a, int b) {
5          int result = a + b;
6          return result; // sends value back to caller
7      }
8
9      // void means no return value
10     public static void greet(String name) {
11         System.out.println("Hello, " + name + "!");
12     }
13
14     public static void main(String[] args) {
15         int sum = add(10, 20); // call the method
16         System.out.println("Sum = " + sum); // Sum = 30
17
18         greet("Priya"); // Hello, Priya!
19         greet("Arjun"); // Hello, Arjun!
20     }
21 }
```

Listing 2.9: Defining and calling methods

Time and Space Complexity

3.1 Why Complexity Matters

Two programs that produce the same correct output can behave very differently when the input is large. Understanding complexity tells you *before coding* whether your approach will be fast enough.

Big-O Notation

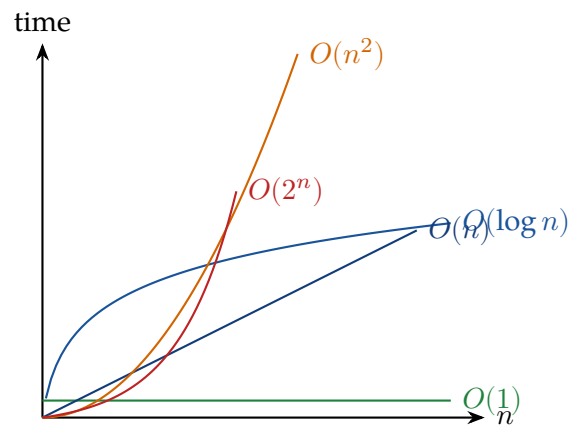
Big-O describes the **worst-case growth rate** of an algorithm as input size $n \rightarrow \infty$. It ignores constant factors and smaller terms.

$$T(n) = 5n^2 + 3n + 100 \implies O(n^2)$$

Think of it as: “As n grows very large, how does the running time grow?”

3.2 Common Complexity Classes

Notation	Name	Classic Example	n = 1000 (ops)
$O(1)$	Constant	Array index access	1
$O(\log n)$	Logarithmic	Binary Search	≈ 10
$O(n)$	Linear	Linear Search	1,000
$O(n \log n)$	Linearithmic	Merge Sort	$\approx 10,000$
$O(n^2)$	Quadratic	Bubble Sort	1,000,000
$O(n^3)$	Cubic	Triple-nested loop	10^9
$O(2^n)$	Exponential	All subsets	$\approx 10^{301}$



3.3 Rules for Analysing Code

1. **Any single statement** (assignment, print, return) $\rightarrow O(1)$
2. **A loop running n times** with $O(1)$ body $\rightarrow O(n)$
3. **Two nested loops, each n times** $\rightarrow O(n^2)$
4. **Two sequential (non-nested) blocks:** add them, keep the dominant term
5. **Input is halved each step** (binary search, etc.) $\rightarrow O(\log n)$

```

1  // O(1): single operation, no loop
2  int getFirst(int[] arr) {
3      return arr[0];
4  }
5
6  // O(n): one loop over n elements
7  int sumArray(int[] arr) {
8      int sum = 0; // O(1)
9      for (int i = 0; i < arr.length; i++) // n iterations
10         sum += arr[i]; // O(1) per iteration
11     return sum; // O(1)
12 } // Total: O(n)
13
14 // O(n^2): nested loops
15 void printAllPairs(int[] arr) {
16     int n = arr.length;
17     for (int i = 0; i < n; i++) // n iterations
18         for (int j = 0; j < n; j++) // n iterations each
19             System.out.print(arr[i] + "," + arr[j] + " ");
20 } // Total: n * n = O(n^2)

```

Listing 3.1: Identifying complexity from code structure

3.4 Space Complexity

Space Complexity

The amount of **extra memory** an algorithm uses, beyond its input.

- $O(1)$ — Only a fixed number of variables (int, double, etc.)
- $O(n)$ — An extra array or list of size n
- $O(n^2)$ — A 2-D matrix of size $n \times n$

The input itself is not counted — only the *additional* memory you allocate.

Interview Corner

- Always mention **both** Time **and** Space complexity
- “ $O(n^2)$ solution” is acceptable when $n \leq 1000$
- For $n \leq 10^6$, aim for $O(n \log n)$ or better
- When asked “Can you optimise?” — the interviewer usually wants $O(n)$
- Trading space for time (and vice versa) is a valid interview discussion

Basic Mathematics for DSA

These problems appear directly in DSA interviews and also build the maths intuition needed for harder problems.

4.1 Count Digits of a Number

Problem: Given integer n , count how many digits it has.

Idea: Keep dividing by 10 until the number becomes 0. Each division removes one digit.

```
1 public class CountDigits {
2     public static int countDigits(int n) {
3         if (n == 0) return 1;           // special case: 0 has 1 digit
4         int count = 0;
5         n = Math.abs(n);                // handle negatives
6
7         while (n > 0) {
8             n = n / 10;                 // remove last digit
9             count++;                    // count this digit
10        }
11        return count;
12    }
13
14    public static void main(String[] args) {
15        System.out.println(countDigits(12345)); // 5
16        System.out.println(countDigits(0));    // 1
17        System.out.println(countDigits(-987)); // 3
18    }
19 }
```

Listing 4.1: Count Digits — Loop method

Dry Run: countDigits(12345)

Iteration	n (before)	n = n / 10	count
1	12345	1234	1
2	1234	123	2
3	123	12	3
4	12	1	4
5	1	0	5
n = 0, loop ends			5

Result: 5 digits**Complexity Analysis**Time: $O(\log_{10} n)$ — one iteration per digit Space: $O(1)$

4.2 Reverse a Number

Idea: Extract the last digit using $\% 10$, append it to the result, remove it with $/ 10$.

```

1 public class ReverseNumber {
2     public static int reverse(int n) {
3         int reversed = 0;
4         n = Math.abs(n);           // handle negatives
5
6         while (n > 0) {
7             int lastDigit = n % 10;           // extract last digit
8             reversed = reversed * 10 + lastDigit; // shift and append
9             n = n / 10;                       // remove last digit
10        }
11        return reversed;
12    }
13
14    public static void main(String[] args) {
15        System.out.println(reverse(12345)); // 54321
16        System.out.println(reverse(100));   // 1
17    }
18 }

```

Listing 4.2: Reverse a Number

Dry Run: reverse(1234)

Step	n	lastDigit (n % 10)	reversed	n after / 10
1	1234	4	$0 \times 10 + 4 = 4$	123
2	123	3	$4 \times 10 + 3 = 43$	12
3	12	2	$43 \times 10 + 2 = 432$	1
4	1	1	$432 \times 10 + 1 = 4321$	0

Result: 4321

4.3 Palindrome Number

A number is a **palindrome** if it reads the same forwards and backwards (e.g., 121, 1221, 12321).

```

1 public class PalindromeCheck {
2     public static boolean isPalindrome(int n) {
3         if (n < 0) return false;           // negatives are never palindromes
4
5         int original = n;
6         int reversed = 0;
7
8         while (n > 0) {
9             reversed = reversed * 10 + (n % 10);
10            n = n / 10;
11        }
12        return original == reversed;
13    }
14
15    public static void main(String[] args) {
16        System.out.println(isPalindrome(121)); // true
17        System.out.println(isPalindrome(123)); // false
18        System.out.println(isPalindrome(1221)); // true
19    }
20 }

```

Listing 4.3: Check Palindrome Number

Complexity AnalysisTime: $O(\log_{10} n)$ Space: $O(1)$

4.4 Factorial of a Number

$n! = n \times (n - 1) \times (n - 2) \times \dots \times 2 \times 1$, with $0! = 1$.

```

1 public class Factorial {
2     public static long factorial(int n) {
3         if (n < 0) return -1;           // factorial undefined for negatives
4         long result = 1;
5         for (int i = 2; i <= n; i++) {
6             result = result * i;
7         }
8         return result;
9     }
10
11     public static void main(String[] args) {
12         System.out.println(factorial(5)); // 120   (5*4*3*2*1)
13         System.out.println(factorial(0)); // 1
14         System.out.println(factorial(10)); // 3628800
15     }
16 }

```

Listing 4.4: Factorial — Iterative approach

Use long for Factorial

int overflows at $n = 13$ ($13! = 6,227,020,800 > 2.1\text{B}$). Use long to handle up to $n = 20$. For $n > 20$, use BigInteger.

4.5 GCD and LCM

GCD (Greatest Common Divisor) — the largest integer that divides both a and b without remainder.

LCM (Lowest Common Multiple) — the smallest positive integer divisible by both.

Relationship: $\text{LCM}(a, b) \times \text{GCD}(a, b) = a \times b$

Euclidean Algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \bmod b)$, stop when $b = 0$.

```

1 public class GcdLcm {
2     public static int gcd(int a, int b) {
3         while (b != 0) {
4             int temp = b;
5             b = a % b;           // b becomes the remainder
6             a = temp;           // a becomes the old b
7         }
8         return a;               // when b = 0, a holds the GCD
9     }
10
11     public static int lcm(int a, int b) {
12         // Divide first to avoid integer overflow

```



```

13     return (a / gcd(a, b)) * b;
14 }
15
16 public static void main(String[] args) {
17     System.out.println("GCD(12, 8) = " + gcd(12, 8)); // 4
18     System.out.println("LCM(12, 8) = " + lcm(12, 8)); // 24
19 }
20 }

```

Listing 4.5: GCD and LCM using Euclidean Algorithm

Dry Run: gcd(12, 8)

Step	a	b	b = a % b
1	12	8	12%8 = 4
2	8	4	8%4 = 0
3	4	0	Loop ends

Result: GCD = 4, so LCM = $(12/4) \times 8 = 24$

Complexity Analysis

Time: $O(\log \min(a, b))$ — the Euclidean algorithm is very fast Space: $O(1)$

4.6 Prime Number Check

A number n is **prime** if it has exactly two divisors: 1 and itself. (1 is not prime; 2 is the only even prime.)

```

1 public class PrimeCheck {
2     public static boolean isPrime(int n) {
3         if (n <= 1) return false;           // 0 and 1 are not prime
4         if (n == 2) return true;           // 2 is prime
5         if (n % 2 == 0) return false;      // all other even numbers: not
6                                           prime
7
8         // Check odd divisors from 3 up to sqrt(n)
9         for (int i = 3; i * i <= n; i += 2) {
10             if (n % i == 0) return false; // found a divisor
11         }
12         return true;
13     }
14
15     public static void main(String[] args) {
16         System.out.println(isPrime(7));    // true
17     }
18 }

```

```
16     System.out.println(isPrime(12));    // false
17     System.out.println(isPrime(97));    // true
18 }
19 }
```

Listing 4.6: Is Prime? — Optimised $O(\sqrt{n})$ check

Why check only up to $\sqrt{n}$?

If n has a divisor $d > \sqrt{n}$, then $n/d < \sqrt{n}$ is also a divisor — and we would have already found it. So we never need to check beyond $\sqrt{n}$.

Example: $n = 36$. Divisors: 1, 2, 3, 4, 6, 9, 12, 18, 36. $\sqrt{36} = 6$. All divisors > 6 pair with one ≤ 6 .

Complexity Analysis

Naive: $O(n)$ Optimised: $O(\sqrt{n})$. For $n = 10^{12}$: from 10^{12} ops down to 10^6 ops.

1-D Arrays

Arrays are the single most important topic for DSA interviews. Nearly every other data structure (stack, queue, heap, hash table) is implemented using arrays internally.

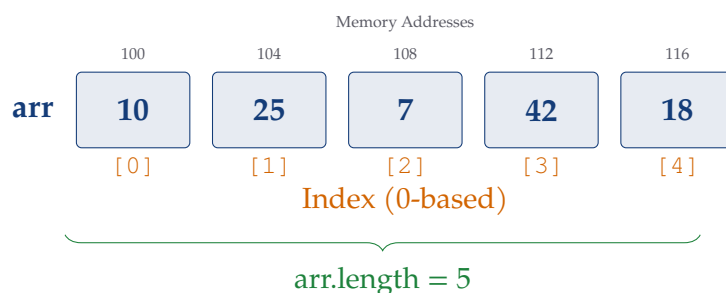
5.1 What is an Array?

Array

An **array** is a collection of elements of the **same data type**, stored in **contiguous (adjacent) memory locations**, each accessible in $O(1)$ time via a 0-based index.

5.1.1 Memory Representation

```
int[ ] arr = { 10, 25, 7, 42, 18 };
```



Key Properties:

- **Fixed size** — Size is set at creation; cannot grow or shrink
- **0-indexed** — First element is at index 0; last is at index `arr.length - 1`
- **$O(1)$ random access** — Any element can be read or written in constant time
- **Same type** — All elements share one data type
- **Contiguous memory** — Elements sit side-by-side in RAM (cache-friendly)

5.2 Declaring and Initialising Arrays

```
1 public class ArrayDeclaration {
2     public static void main(String[] args) {
3
4         // WAY 1: Declare size only -- JVM fills with default values
5         // int -> 0, double -> 0.0, boolean -> false, String -> null
6         int[] arr1 = new int[5];           // {0, 0, 0, 0, 0}
7
8         // WAY 2: Declare with values -- compiler counts the size
9         int[] arr2 = {10, 20, 30, 40, 50};
10
11        // WAY 3: new keyword with values (same result as Way 2)
12        int[] arr3 = new int[]{10, 20, 30, 40, 50};
13
14        // ----- Accessing elements -----
15        System.out.println(arr2[0]);       // 10 (first element, index 0)
16        System.out.println(arr2[4]);       // 50 (last element, index 4)
17
18        // arr2[5] -> ArrayIndexOutOfBoundsException (out of range!)
19
20        // ----- Modifying an element -----
21        arr2[2] = 99;
22        System.out.println(arr2[2]);       // 99
23
24        // ----- Length (PROPERTY, not method - no parentheses!)
25        // -----
26        System.out.println("Size = " + arr2.length); // 5
27
28        // ----- Last element -----
29        System.out.println("Last = " + arr2[arr2.length - 1]); // 50
30    }
```

Listing 5.1: All ways to declare and initialise arrays in Java

Most Common Array Mistakes

`arr.length` is a **property** — no parentheses. Do not write `arr.length()`.
Valid indices: 0 to `arr.length - 1`. Accessing `arr[arr.length]` throws `ArrayIndexOutOfBoundsException`.

5.3 Traversing an Array

```
1 public class ArrayTraversal {
2     public static void main(String[] args) {
3         int[] arr = {10, 20, 30, 40, 50};
```

```
4
5      // WAY 1: Traditional for loop
6      //   Use when: you need the INDEX (most DSA problems)
7      System.out.print("for loop : ");
8      for (int i = 0; i < arr.length; i++) {
9          System.out.print(arr[i] + " ");
10     }
11     System.out.println();           // Output: 10 20 30 40 50
12
13     // WAY 2: Enhanced for-each loop
14     //   Use when: you only need VALUES, not the index (read-only)
15     System.out.print("for-each : ");
16     for (int x : arr) {
17         System.out.print(x + " ");
18     }
19     System.out.println();           // Output: 10 20 30 40 50
20
21     // WAY 3: while loop
22     //   Use when: the traversal condition is complex
23     System.out.print("while      : ");
24     int i = 0;
25     while (i < arr.length) {
26         System.out.print(arr[i] + " ");
27         i++;
28     }
29     System.out.println();           // Output: 10 20 30 40 50
30 }
31 }
```

Listing 5.2: Three ways to traverse — when to use each

5.4 Sum of Array Elements

```
1 public class ArraySum {
2     public static int sumArray(int[] arr) {
3         int sum = 0;
4         for (int i = 0; i < arr.length; i++) {
5             sum = sum + arr[i];    // add current element to running total
6         }
7         return sum;
8     }
9
10    public static void main(String[] args) {
11        int[] arr = {3, 1, 4, 1, 5, 9, 2, 6};
12        System.out.println("Sum = " + sumArray(arr));    // 31
13    }
14 }
```

Listing 5.3: Sum of all elements — accumulator pattern

Dry Run: arr = {3, 1, 4, 1, 5}

i	arr[i]	sum (before)	sum (after)
0	3	0	3
1	1	3	4
2	4	4	8
3	1	8	9
4	5	9	14

Result: 14

Complexity Analysis

Time: $O(n)$ — visit every element once Space: $O(1)$ — only `sum` variable

5.5 Largest and Smallest Element

```

1 public class MaxMin {
2     public static int findMax(int[] arr) {
3         int max = arr[0];           // assume first element is max
4         for (int i = 1; i < arr.length; i++) {
5             if (arr[i] > max) {
6                 max = arr[i];       // update when a larger value found
7             }
8         }
9         return max;
10    }
11
12    public static int findMin(int[] arr) {
13        int min = arr[0];
14        for (int i = 1; i < arr.length; i++) {
15            if (arr[i] < min) {
16                min = arr[i];
17            }
18        }
19        return min;
20    }
21
22    public static void main(String[] args) {
23        int[] arr = {64, 34, 25, 12, 22, 11, 90};
24        System.out.println("Max = " + findMax(arr)); // 90
25        System.out.println("Min = " + findMin(arr)); // 11
26    }
27 }

```

Listing 5.4: Find Max and Min in a single pass each

Dry Run: findMax on {64, 34, 25, 12, 22, 11, 90}

i	arr[i]	max	Updated?
init	—	64	max = arr[0] = 64
1	34	64	34 > 64? No
2	25	64	25 > 64? No
3	12	64	12 > 64? No
4	22	64	22 > 64? No
5	11	64	11 > 64? No
6	90	90	90 > 64? Yes → max = 90

Result: max = 90**Complexity Analysis**Time: $O(n)$ Space: $O(1)$

5.6 Second Largest Element

Find the second largest without sorting — in a single pass.

```

1 public class SecondLargest {
2     public static int secondLargest(int[] arr) {
3         if (arr.length < 2) return -1;    // not enough elements
4
5         int first = Integer.MIN_VALUE;    // will hold the largest
6         int second = Integer.MIN_VALUE;   // will hold second largest
7
8         for (int i = 0; i < arr.length; i++) {
9             if (arr[i] > first) {
10                second = first;             // old largest drops to second place
11                first = arr[i];             // new largest found
12            } else if (arr[i] > second && arr[i] != first) {
13                second = arr[i];            // update second (skip duplicates of
14                                           // first)
15            }
16        }
17        if (second == Integer.MIN_VALUE) return -1; // all elements identical
18        return second;
19    }
20 }

```

```

19
20     public static void main(String[] args) {
21         int[] arr = {12, 35, 1, 10, 34, 1};
22         System.out.println(secondLargest(arr)); // 34
23     }
24 }

```

Listing 5.5: Second Largest — Single Pass $O(n)$

Dry Run: arr = {12, 35, 1, 10, 34, 1}

i	arr[i]	first	second
init	—	MIN_VALUE	MIN_VALUE
0	12	12	MIN_VALUE
1	35	35	12
2	1	35	12
3	10	35	12
4	34	35	34
5	1	35	34

Result: 34

Complexity Analysis

Time: $O(n)$ — single pass Space: $O(1)$

5.7 Linear Search

Problem: Search for a target value. Return its index, or -1 if not found.

```

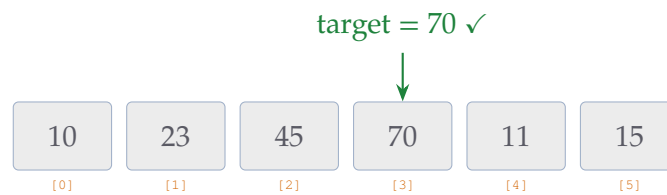
1  public class LinearSearch {
2      public static int linearSearch(int[] arr, int target) {
3          for (int i = 0; i < arr.length; i++) {
4              if (arr[i] == target) {
5                  return i; // found! return the index
6              }
7          }
8          return -1; // target not in array
9      }
10
11     public static void main(String[] args) {
12         int[] arr = {10, 23, 45, 70, 11, 15};
13         System.out.println(linearSearch(arr, 70)); // 3
14         System.out.println(linearSearch(arr, 99)); // -1

```



```
15     }  
16 }
```

Listing 5.6: Linear Search — scan left to right



Complexity Analysis

Best: $O(1)$ (found at index 0) Worst: $O(n)$ (last or not present) Space: $O(1)$

5.8 Check if Array is Sorted

```
1 public class IsSorted {  
2     public static boolean isSorted(int[] arr) {  
3         // Compare every adjacent pair; if any pair violates order, return  
4         // false  
5         for (int i = 0; i < arr.length - 1; i++) {  
6             if (arr[i] > arr[i + 1]) {  
7                 return false;    // found a descending pair -- not sorted  
8             }  
9         }  
10        return true;           // no violations found  
11    }  
12  
13    public static void main(String[] args) {  
14        int[] a = {1, 2, 3, 4, 5};  
15        int[] b = {1, 3, 2, 4, 5};    // 3 > 2: violation at index 1-2  
16        System.out.println(isSorted(a));    // true  
17        System.out.println(isSorted(b));    // false  
18    }  
19 }
```

Listing 5.7: Is the array sorted in ascending order?

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

5.9 Reverse an Array

Approach: Two-pointer technique. Place one pointer at the start (`left`) and one at the end (`right`). Swap and move both inward until they meet.

```

1 public class ReverseArray {
2     public static void reverse(int[] arr) {
3         int left = 0;
4         int right = arr.length - 1;
5
6         while (left < right) {
7             // Swap arr[left] and arr[right]
8             int temp = arr[left];
9             arr[left] = arr[right];
10            arr[right] = temp;
11
12            left++;    // move left pointer rightward
13            right--;   // move right pointer leftward
14        }
15    }
16
17    public static void main(String[] args) {
18        int[] arr = {1, 2, 3, 4, 5};
19        System.out.print("Before: ");
20        for (int x : arr) System.out.print(x + " ");
21
22        reverse(arr);
23
24        System.out.print("\nAfter: ");
25        for (int x : arr) System.out.print(x + " ");
26        // Output Before: 1 2 3 4 5
27        // Output After:  5 4 3 2 1
28    }
29 }

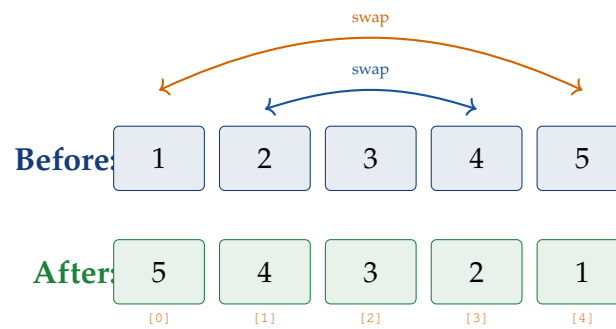
```

Listing 5.8: Reverse Array — Two Pointer, in-place

Dry Run: reverse({1, 2, 3, 4, 5})

Step	left	right	Swap	Array State
Start	0	4	—	{1, 2, 3, 4, 5}
1	0	4	arr[0] ↔ arr[4]	{5, 2, 3, 4, 1}
2	1	3	arr[1] ↔ arr[3]	{5, 4, 3, 2, 1}
3	2	2	left >= right: stop	{5, 4, 3, 2, 1}

Result: {5, 4, 3, 2, 1}



Complexity Analysis

Time: $O(n)$ — $\lfloor n/2 \rfloor$ swaps Space: $O(1)$ — in-place, only temp

5.10 Count Odd Numbers in Array

```

1 public class CountOdd {
2     public static int countOdd(int[] arr) {
3         int count = 0;
4         for (int i = 0; i < arr.length; i++) {
5             if (arr[i] % 2 != 0) { // odd: remainder when divided by 2 is 1
6                 count++;
7             }
8         }
9         return count;
10    }
11
12    public static void main(String[] args) {
13        int[] arr = {1, 2, 3, 4, 5, 6, 7, 8};
14        System.out.println("Odd count = " + countOdd(arr)); // 4
15    }
16 }

```

Listing 5.9: Count all odd elements

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

5.11 Left Rotate Array by One

```

1 public class RotateByOne {
2     public static void rotateByOne(int[] arr) {

```

```

3      int n      = arr.length;
4      int first = arr[0];           // save the first element
5
6      // shift every element one index to the left
7      for (int i = 0; i < n - 1; i++) {
8          arr[i] = arr[i + 1];
9      }
10     arr[n - 1] = first;           // place saved element at the end
11 }
12
13 public static void main(String[] args) {
14     int[] arr = {1, 2, 3, 4, 5};
15     System.out.print("Before: ");
16     for (int x : arr) System.out.print(x + " ");    // 1 2 3 4 5
17
18     rotateByOne(arr);
19
20     System.out.print("\nAfter: ");
21     for (int x : arr) System.out.print(x + " ");    // 2 3 4 5 1
22 }
23 }

```

Listing 5.10: Left Rotate by One Position

Dry Run: arr = {1, 2, 3, 4, 5}

Action	Array State	Notes
Initial	{1, 2, 3, 4, 5}	first = 1 (saved)
After shift loop	{2, 3, 4, 5, 5}	each element moved left
After arr[4] = 1	{2, 3, 4, 5, 1}	saved element placed at end

Result: {2, 3, 4, 5, 1}

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

5.12 Left Rotate by K Places

Approach: Triple Reverse Trick

To rotate left by k :

1. Reverse the first k elements
2. Reverse the remaining $n - k$ elements

3. Reverse the entire array

```

1 public class RotateByK {
2     // Helper: reverse arr[start..end] in-place
3     private static void reverse(int[] arr, int start, int end) {
4         while (start < end) {
5             int temp      = arr[start];
6             arr[start]    = arr[end];
7             arr[end]      = temp;
8             start++;
9             end--;
10        }
11    }
12
13    public static void rotateLeft(int[] arr, int k) {
14        int n = arr.length;
15        k = k % n;           // e.g., k=7, n=5 -> k=2 (same effect)
16        if (k == 0) return; // nothing to rotate
17
18        reverse(arr, 0, k - 1); // Step 1: reverse first k elements
19        reverse(arr, k, n - 1); // Step 2: reverse remaining n-k elements
20        reverse(arr, 0, n - 1); // Step 3: reverse the whole array
21    }
22
23    public static void main(String[] args) {
24        int[] arr = {1, 2, 3, 4, 5};
25        rotateLeft(arr, 2);
26        for (int x : arr) System.out.print(x + " ");
27        // Output: 3 4 5 1 2
28    }
29 }

```

Listing 5.11: Left Rotate by K — Triple Reverse, $O(n)$ time $O(1)$ space

Dry Run: arr = {1, 2, 3, 4, 5}, k = 2

Step	Array State
Initial	{1, 2, 3, 4, 5}
Reverse arr[0..1]	{2, 1, 3, 4, 5}
Reverse arr[2..4]	{2, 1, 5, 4, 3}
Reverse arr[0..4] (whole)	{3, 4, 5, 1, 2}

Result: {3, 4, 5, 1, 2} — elements at positions 0 and 1 moved to the end.

Complexity Analysis

Time: $O(n)$ — three reversals each $O(n)$ Space: $O(1)$ — completely in-place

Interview Corner

Also know the **extra-array approach**: copy the last $n - k$ elements to a temp array, then the first k elements, then copy back. It is $O(n)$ time but $O(n)$ space — worse in space. The triple-reverse trick is the optimal solution.

5.13 Maximum Consecutive Ones

Problem: Given a binary array (only 0s and 1s), find the length of the longest consecutive run of 1s.

```
1 public class MaxConsecutiveOnes {
2     public static int maxOnes(int[] arr) {
3         int maxCount = 0;    // best streak seen so far
4         int count     = 0;    // current streak length
5
6         for (int i = 0; i < arr.length; i++) {
7             if (arr[i] == 1) {
8                 count++;           // extend the streak
9                 maxCount = Math.max(maxCount, count); // update best
10            } else {
11                count = 0;          // 0 breaks the streak; reset
12            }
13        }
14        return maxCount;
15    }
16
17    public static void main(String[] args) {
18        int[] arr = {1, 1, 0, 1, 1, 1, 0, 1};
19        System.out.println(maxOnes(arr)); // 3
20    }
21 }
```

Listing 5.12: Maximum Consecutive Ones — streak counter

Dry Run: `arr = {1, 1, 0, 1, 1, 1, 0, 1}`

i	arr[i]	count	maxCount	Note
0	1	1	1	streak starts
1	1	2	2	streak extends
2	0	0	2	reset on 0
3	1	1	2	new streak
4	1	2	2	extends
5	1	3	3	new best!
6	0	0	3	reset on 0
7	1	1	3	streak, max stays 3

Result: 3

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

5.14 Move Zeros to End

Problem: Move all zeros to the end while preserving the **relative order** of non-zero elements.

```

1 public class MoveZeros {
2     public static void moveZeros(int[] arr) {
3         int j = 0;    // j = index where the next non-zero element goes
4
5         // Pass 1: push all non-zeros to the front
6         for (int i = 0; i < arr.length; i++) {
7             if (arr[i] != 0) {
8                 arr[j] = arr[i];    // place non-zero at position j
9                 j++;
10            }
11            // if arr[i] == 0: skip it
12        }
13
14        // Pass 2: fill remaining positions (j to end) with zeros
15        while (j < arr.length) {
16            arr[j] = 0;
17            j++;
18        }
19    }
20
21    public static void main(String[] args) {

```

```

22     int[] arr = {0, 1, 0, 3, 12};
23     moveZeros(arr);
24     for (int x : arr) System.out.print(x + " ");
25     // Output: 1 3 12 0 0
26 }
27 }

```

Listing 5.13: Move Zeros to End — Two Pointer, $O(n)$ $O(1)$ Dry Run: $\text{arr} = \{0, 1, 0, 3, 12\}$

i	arr[i]	j (before)	j (after)	Action
0	0	0	0	Skip (zero)
1	1	0	1	$\text{arr}[0] = 1$
2	0	1	1	Skip (zero)
3	3	1	2	$\text{arr}[1] = 3$
4	12	2	3	$\text{arr}[2] = 12$
Fill: $\text{arr}[3] = 0, \text{arr}[4] = 0 \rightarrow \{1, 3, 12, 0, 0\}$				

Complexity AnalysisTime: $O(n)$ Space: $O(1)$

5.15 Remove Duplicates from Sorted Array

Problem: Given a **sorted** array, remove duplicates in-place. Return the count of unique elements.

```

1  public class RemoveDuplicates {
2      public static int removeDuplicates(int[] arr) {
3          if (arr.length == 0) return 0;
4
5          int j = 0;    // j points to the last unique element placed
6
7          for (int i = 1; i < arr.length; i++) {
8              if (arr[i] != arr[j]) {    // found a new unique element
9                  j++;                  // move j forward
10                 arr[j] = arr[i];      // place it at the next unique position
11             }
12             // if arr[i] == arr[j]: it's a duplicate -- skip it
13         }
14         return j + 1;    // unique count: indices 0 through j
15     }

```



```

16
17     public static void main(String[] args) {
18         int[] arr = {1, 1, 2, 2, 3, 4, 4, 5};
19         int uniqueLen = removeDuplicates(arr);
20
21         System.out.print("Unique elements: ");
22         for (int i = 0; i < uniqueLen; i++) {
23             System.out.print(arr[i] + " ");
24         }
25         // Output: 1 2 3 4 5
26     }
27 }

```

Listing 5.14: Remove Duplicates from Sorted Array — Two Pointer

Dry Run: arr = {1, 1, 2, 2, 3}

i	arr[i]	arr[j]	j	Note
init	—	arr[0]=1	0	Initial state
1	1	1	0	Duplicate, skip
2	2	1	1	Unique! arr[1] = 2
3	2	2	1	Duplicate, skip
4	3	2	2	Unique! arr[2] = 3

Result: uniqueLen = 3, array front = {1, 2, 3}

Only Works on Sorted Arrays!

If the array is unsorted, use a `LinkedHashSet` to preserve order while removing duplicates, or sort first in $O(n \log n)$.

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

5.16 Find the Missing Number

Problem: Given an array of n distinct integers from range $[1, n]$, exactly one number is missing. Find it.

```

1 public class MissingNumber {
2     // METHOD 1: Gauss Sum Formula
3     // Expected sum of 1..n = n*(n+1)/2
4     // Missing = expected - actual

```

```

5     public static int missingBySum(int[] arr) {
6         int n          = arr.length + 1;           // total should be n numbers
7         int expectedSum = n * (n + 1) / 2;         // Gauss formula
8         int actualSum   = 0;
9         for (int x : arr) actualSum += x;
10        return expectedSum - actualSum;
11    }

12
13    // METHOD 2: XOR (preferred -- avoids overflow for very large n)
14    //   x ^ x = 0,   x ^ 0 = x
15    //   XOR all 1..n with all array values; pairs cancel; missing survives
16    public static int missingByXOR(int[] arr) {
17        int n          = arr.length + 1;
18        int xorAll = 0;
19        for (int i = 1; i <= n; i++) xorAll ^= i;    // XOR 1 to n
20        for (int x : arr) xorAll ^= x;             // XOR all array values
21        return xorAll;                             // missing number remains
22    }

23
24    public static void main(String[] args) {
25        int[] arr = {1, 2, 4, 5, 6};              // 3 is missing
26        System.out.println(missingBySum(arr));    // 3
27        System.out.println(missingByXOR(arr));    // 3
28    }
29 }

```

Listing 5.15: Missing Number — Sum Formula and XOR (both shown)

Sum Method: arr = {1, 2, 4, 5, 6}, range [1..6]

] Expected sum = $6 \times 7/2 = 21$
 Actual sum = $1 + 2 + 4 + 5 + 6 = 18$
 Missing = $21 - 18 = 3$

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

Interview Corner

The **XOR method** is preferred in interviews because the Gauss sum can overflow `int` for very large n (e.g., $n = 10^5$ gives sum $\approx 5 \times 10^9$, which overflows 32-bit `int`). Use `long` with the sum method as a safe alternative. XOR has no overflow risk.

5.17 Union of Two Sorted Arrays

Problem: Return a sorted array containing every unique element from both sorted arrays.

```

1  import java.util.ArrayList;
2
3  public class UnionArrays {
4      public static ArrayList<Integer> union(int[] a, int[] b) {
5          ArrayList<Integer> res = new ArrayList<>();
6          int i = 0, j = 0;
7
8          while (i < a.length && j < b.length) {
9              // Skip duplicates already added to result
10             if (!res.isEmpty() && res.get(res.size() - 1) == a[i]) { i++;
11                 continue; }
12             if (!res.isEmpty() && res.get(res.size() - 1) == b[j]) { j++;
13                 continue; }
14
15             if (a[i] < b[j]) res.add(a[i++]);
16             else if (a[i] > b[j]) res.add(b[j++]);
17             else { res.add(a[i]); i++; j++; } // both equal
18         }
19         // Drain remaining elements from a
20         while (i < a.length) {
21             if (res.isEmpty() || res.get(res.size()-1) != a[i]) res.add(a[i]);
22             i++;
23         }
24         // Drain remaining elements from b
25         while (j < b.length) {
26             if (res.isEmpty() || res.get(res.size()-1) != b[j]) res.add(b[j]);
27             j++;
28         }
29         return res;
30     }
31
32     public static void main(String[] args) {
33         int[] a = {1, 2, 3, 4, 5};
34         int[] b = {1, 2, 3, 6, 7};
35         System.out.println(union(a, b)); // [1, 2, 3, 4, 5, 6, 7]
36     }
37 }

```

Listing 5.16: Union of Two Sorted Arrays — Two Pointer, $O(m+n)$ **Complexity Analysis**

Time: $O(m + n)$ — one pass over both Space: $O(m + n)$ for result list

5.18 Intersection of Two Sorted Arrays

Problem: Return elements common to both sorted arrays (no duplicates).

```
1 import java.util.ArrayList;
2
3 public class IntersectionArrays {
4     public static ArrayList<Integer> intersection(int[] a, int[] b) {
5         ArrayList<Integer> res = new ArrayList<>();
6         int i = 0, j = 0;
7
8         while (i < a.length && j < b.length) {
9             if (a[i] == b[j]) {
10                 // Common element -- add only if not already added
11                 if (res.isEmpty() || res.get(res.size()-1) != a[i])
12                     res.add(a[i]);
13                 i++;
14                 j++;
15             } else if (a[i] < b[j]) {
16                 i++; // a[i] is smaller; advance a
17             } else {
18                 j++; // b[j] is smaller; advance b
19             }
20         }
21         return res;
22     }
23
24     public static void main(String[] args) {
25         int[] a = {1, 2, 3, 4, 5};
26         int[] b = {2, 3, 5, 7};
27         System.out.println(intersection(a, b)); // [2, 3, 5]
28     }
29 }
```

Listing 5.17: Intersection of Two Sorted Arrays — Two Pointer

Complexity Analysis

Time: $O(m + n)$ Space: $O(\min(m, n))$ for result

5.19 Two Sum Problem

Problem: Given an integer array and a target, return the **indices** of the two numbers that add up to target.

```
1 import java.util.HashMap;
2
3 public class TwoSum {
4     public static int[] twoSum(int[] arr, int target) {
5         // Map stores: element value -> its index
6         HashMap<Integer, Integer> map = new HashMap<>();
7
8         for (int i = 0; i < arr.length; i++) {
```

```

9      int complement = target - arr[i];    // what we still need
10
11     if (map.containsKey(complement)) {
12         // Found the pair!
13         return new int[]{ map.get(complement), i };
14     }
15     map.put(arr[i], i);    // store current element for future look-up
16 }
17 return new int[]{-1, -1}; // no pair found
18 }
19
20 public static void main(String[] args) {
21     int[] arr = {2, 7, 11, 15};
22     int[] res = twoSum(arr, 9);
23     System.out.println "[" + res[0] + ", " + res[1] + ""];
24     // Output: [0, 1] because arr[0] + arr[1] = 2 + 7 = 9
25 }
26 }

```

Listing 5.18: Two Sum — HashMap approach $O(n)$ time $O(n)$ space

Dry Run: arr = {2, 7, 11, 15}, target = 9

i	arr[i]	complement	In map?	map contents	Action
0	2	$9 - 2 = 7$	No	{}	put(2, 0)
1	7	$9 - 7 = 2$	Yes!	{2:0}	return [0, 1]

Complexity Analysis

Approach	Time	Space
Brute Force (nested loops)	$O(n^2)$	$O(1)$
HashMap (this solution)	$O(n)$	$O(n)$
Two Pointer (sorted array)	$O(n \log n)$ sort + $O(n)$	$O(1)$

Interview Corner

One of the **most-asked** interview questions at every company. Know all three approaches and when to use each:

- HashMap when unsorted, $O(n)$ preferred
- Two Pointer when array is already sorted and $O(1)$ space needed
- Brute force only for very small n or when explaining the naive approach first

5.20 Leaders in an Array

Definition: An element is a **leader** if it is strictly greater than all elements to its right. The rightmost element is always a leader.

```

1  import java.util.ArrayList;
2  import java.util.Collections;
3
4  public class Leaders {
5      public static ArrayList<Integer> findLeaders(int[] arr) {
6          ArrayList<Integer> leaders = new ArrayList<>();
7          int n = arr.length;
8
9          int maxFromRight = arr[n - 1];    // rightmost is always a leader
10         leaders.add(maxFromRight);
11
12         // Scan right to left; a leader must be > every element to its right
13         for (int i = n - 2; i >= 0; i--) {
14             if (arr[i] > maxFromRight) {
15                 maxFromRight = arr[i];
16                 leaders.add(arr[i]);
17             }
18         }
19         Collections.reverse(leaders);    // restore left-to-right order
20         return leaders;
21     }
22
23     public static void main(String[] args) {
24         int[] arr = {16, 17, 4, 3, 5, 2};
25         System.out.println(findLeaders(arr)); // [17, 5, 2]
26     }
27 }

```

Listing 5.19: Leaders in Array — Right-to-Left scan, $O(n)$

Dry Run: arr = {16, 17, 4, 3, 5, 2}

1. Initialise: maxFromRight = 2, leaders = {2}
2. i=4: arr[4]=5. 5 > 2? Yes → max=5, leaders={2, 5}
3. i=3: arr[3]=3. 3 > 5? No
4. i=2: arr[2]=4. 4 > 5? No
5. i=1: arr[1]=17. 17 > 5? Yes → max=17, leaders={2, 5, 17}
6. i=0: arr[0]=16. 16 > 17? No
7. Reverse leaders → {17, 5, 2}

Result: {17, 5, 2}

Complexity Analysis

Time: $O(n)$ Space: $O(k)$ where k = number of leaders

Array Problems — Interview Revision

6.1 Complete Problem Reference Table

Problem	Approach	Time	Core Idea
Sum of elements	Single loop	$O(n)$	Accumulate
Max / Min element	Single loop	$O(n)$	Track running best
Second largest	Single loop	$O(n)$	Two variables: first, second
Linear search	Single loop	$O(n)$	Compare and return index
Check sorted	Loop $n-1$ pairs	$O(n)$	$\text{arr}[i] \leq \text{arr}[i+1]$?
Reverse array	Two pointers	$O(n)$	Swap from both ends
Move zeros to end	Two pointers	$O(n)$	Compact non-zeros, fill zeros
Remove duplicates	Two pointers	$O(n)$	Sorted array only
Find missing number	XOR / Gauss	$O(n)$	Expected — actual sum
Rotate left by 1	Save and shift	$O(n)$	Save first, shift, place last
Rotate left by K	Triple reverse	$O(n)$	Reverse three segments
Max consecutive ones	Single loop	$O(n)$	Streak counter with reset
Two Sum	HashMap	$O(n)$	$\text{complement} = \text{target} - x$
Sort 0s / 1s / 2s	Dutch NF	$O(n)$	low, mid, high pointers (Ch. 8)

Continued on next page

Problem	Approach	Time	Core Idea
Majority element	Boyer-Moore	$O(n)$	Vote cancellation (Ch. 8)
Leaders in array	Right to left	$O(n)$	Track max from right

Table 6.1: 1-D Array Problems — Complete Quick Reference

6.2 The Two Pointer Technique

Two Pointer Pattern

Use two index variables to traverse the array simultaneously. They can move from both ends inward, or both from the left at different speeds.

When to use:

- Array is sorted and you need pairs satisfying a condition
- Removing or moving elements in-place
- Palindrome / symmetry checks
- Subarrays with a running window (sliding window variant)

Typical gain: Reduces $O(n^2)$ brute force to $O(n)$.

6.3 Top Interview Questions

Must-Know Array Problems for Interviews

1. **Kadane's Algorithm** — Maximum subarray sum ($O(n)$)
2. **Find first and last position** of element in sorted array (Binary Search)
3. **Move negatives to left**, positives to right
4. **Merge two sorted arrays** without extra space
5. **Find the duplicate** in array of size $n + 1$, values $[1, n]$
6. **Pascal's Triangle** — print row n
7. **Rearrange alternating** positive and negative numbers
8. **Equilibrium index** — where left sum equals right sum
9. **3-Sum problem** — find all triplets summing to zero
10. **Minimum jumps** to reach end of array

6.4 Common Edge Cases to Check

Always Test These Cases in Interviews

- **Empty array** (`arr.length == 0`) — return early or return `-1`
- **Single element** (`arr.length == 1`) — often needs special handling
- **All elements same** — e.g., `{5, 5, 5, 5}` — duplicates everywhere
- **Already sorted / reverse sorted** — check both extremes
- **Negative numbers** — especially for max/min and sum problems
- **Integer overflow** — use `long` when summing large arrays
- **`k = 0` or `k = n`** — for rotation problems (no change expected)

Sorting Algorithms

Sorting is arranging array elements in a specific order (ascending or descending). It is a prerequisite for Binary Search and many other algorithms. Know these algorithms cold — both the code and the complexity.

Algorithm	Best	Average	Worst	Space	Stable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(1)$	No
Radix Sort	$O(dn)$	$O(d(n + b))$	$O(d(n + b))$	$O(n + b)$	Yes

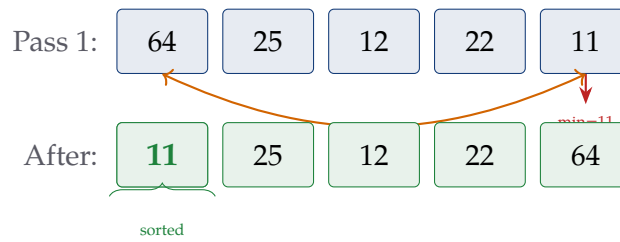
d = number of digits in max element b = base (10 for decimal)

Stable Sort?

A sort is **stable** if elements with equal values maintain their original relative order. Stability matters when sorting objects by multiple keys (e.g., sort students by grade, then by name within the same grade).

7.1 Selection Sort

Idea: Find the minimum element in the unsorted region and swap it to the front. Repeat.



```

1 public class SelectionSort {
2     public static void selectionSort(int[] arr) {
3         int n = arr.length;
4
5         for (int i = 0; i < n - 1; i++) {
6             // Find the index of the minimum element in arr[i..n-1]
7             int minIndex = i;
8             for (int j = i + 1; j < n; j++) {
9                 if (arr[j] < arr[minIndex]) {
10                     minIndex = j; // found a smaller element
11                 }
12             }
13             // Swap the minimum element to position i
14             int temp = arr[minIndex];
15             arr[minIndex] = arr[i];
16             arr[i] = temp;
17         }
18     }
19
20     public static void main(String[] args) {
21         int[] arr = {64, 25, 12, 22, 11};
22         selectionSort(arr);
23         for (int x : arr) System.out.print(x + " ");
24         // Output: 11 12 22 25 64
25     }
26 }

```

Listing 7.1: Selection Sort

Dry Run: arr = {64, 25, 12, 22, 11}

Pass (i)	Unsorted Region	Minimum Found	Array After Swap
0	{64,25,12,22,11}	11 at index 4	{11,25,12,22,64}
1	{25,12,22,64}	12 at index 2	{11,12,25,22,64}
2	{25,22,64}	22 at index 3	{11,12,22,25,64}
3	{25,64}	25 at index 3	{11,12,22,25,64}

Result: {11, 12, 22, 25, 64}

Complexity Analysis

Time: $O(n^2)$ always (even if already sorted — it still scans) Space: $O(1)$ Stable: No

7.2 Bubble Sort

Idea: Repeatedly compare adjacent elements and swap if out of order. After each pass, the largest unsorted element “bubbles” to its correct position at the end.

```

1 public class BubbleSort {
2     public static void bubbleSort(int[] arr) {
3         int n = arr.length;
4
5         for (int i = 0; i < n - 1; i++) {
6             boolean swapped = false;    // optimisation: detect already-sorted
              array
7
8             // Inner loop: push the largest in arr[0..n-i-1] to its final
              position
9             for (int j = 0; j < n - i - 1; j++) {
10                if (arr[j] > arr[j + 1]) {
11                    // Swap adjacent elements
12                    int temp    = arr[j];
13                    arr[j]      = arr[j + 1];
14                    arr[j + 1]  = temp;
15                    swapped = true;
16                }
17            }
18            // If no swap happened in this pass, array is already sorted
19            if (!swapped) break;
20        }
21    }
22
23    public static void main(String[] args) {
24        int[] arr = {5, 1, 4, 2, 8};
25        bubbleSort(arr);
26        for (int x : arr) System.out.print(x + " ");
27        // Output: 1 2 4 5 8
28    }
29 }

```

Listing 7.2: Bubble Sort with early-exit optimisation

Dry Run: arr = {5, 1, 4, 2, 8}

Pass (i)	Array Before	Comparisons & Swaps	Array After
0	{5,1,4,2,8}	5>1 swap; 5>4 swap; 5>2 swap; 5<8 ok	{1,4,2,5,8}
1	{1,4,2,5,8}	1<4 ok; 4>2 swap; 4<5 ok	{1,2,4,5,8}
2	{1,2,4,5,8}	1<2 ok; 2<4 ok (no swaps)	{1,2,4,5,8}
swapped=false in pass 2 → early exit!			

Result: {1, 2, 4, 5, 8}

Complexity Analysis

Best: $O(n)$ (sorted input, early exit) Worst/Average: $O(n^2)$ Space: $O(1)$ Stable: Yes

7.3 Insertion Sort

Idea: Build a sorted region from left to right. For each new element, insert it into its correct position in the sorted region — just like sorting playing cards in your hand.

```

1 public class InsertionSort {
2     public static void insertionSort(int[] arr) {
3         int n = arr.length;
4
5         for (int i = 1; i < n; i++) {
6             int key = arr[i];    // the element to insert into sorted region
7             int j = i - 1;    // start of the sorted region's right end
8
9             // Shift all sorted elements greater than key one position right
10            while (j >= 0 && arr[j] > key) {
11                arr[j + 1] = arr[j];
12                j--;
13            }
14            // Place key at the correct position
15            arr[j + 1] = key;
16        }
17    }
18
19    public static void main(String[] args) {
20        int[] arr = {12, 11, 13, 5, 6};
21        insertionSort(arr);
22        for (int x : arr) System.out.print(x + " ");
23        // Output: 5 6 11 12 13
24    }
25 }

```

Listing 7.3: Insertion Sort

Dry Run: `arr = {12, 11, 13, 5, 6}`

i	key	Shifting	Array After Insert
1	11	$12 > 11$: shift 12 right	{11, 12, 13, 5, 6}
2	13	$12 \not> 13$: no shift	{11, 12, 13, 5, 6}
3	5	13,12,11 all > 5 : shift all	{5, 11, 12, 13, 6}
4	6	13,12,11 > 6 : shift; 5 < 6 : stop	{5, 6, 11, 12, 13}

Result: {5, 6, 11, 12, 13}

Complexity Analysis

Best: $O(n)$ (already sorted) Worst/Average: $O(n^2)$ Space: $O(1)$ Stable: Yes
Insertion Sort is preferred for small arrays ($n \leq 20$) and nearly-sorted data.

7.4 Merge Sort

Idea: Divide and Conquer. Recursively split the array into halves until size 1, then *merge* the sorted halves back together.

```

1 public class MergeSort {
2     // Merge two sorted halves: arr[left..mid] and arr[mid+1..right]
3     static void merge(int[] arr, int left, int mid, int right) {
4         int n1 = mid - left + 1;
5         int n2 = right - mid;
6
7         int[] L = new int[n1];    // temp left half
8         int[] R = new int[n2];    // temp right half
9
10        for (int i = 0; i < n1; i++) L[i] = arr[left + i];
11        for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
12
13        int i = 0, j = 0, k = left;
14        while (i < n1 && j < n2) {
15            if (L[i] <= R[j]) arr[k++] = L[i++];
16            else arr[k++] = R[j++];
17        }
18        while (i < n1) arr[k++] = L[i++];    // copy remaining left
19        while (j < n2) arr[k++] = R[j++];    // copy remaining right
20    }
21 }

```

```

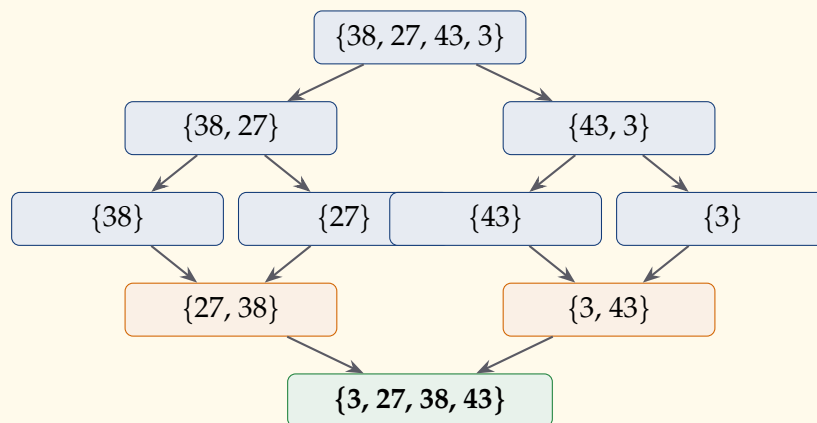
22  static void mergeSort(int[] arr, int left, int right) {
23      if (left < right) {
24          int mid = (left + right) / 2;
25          mergeSort(arr, left, mid);           // sort left half
26          mergeSort(arr, mid + 1, right);      // sort right half
27          merge(arr, left, mid, right);        // merge sorted halves
28      }
29  }

30
31  public static void main(String[] args) {
32      int[] arr = {38, 27, 43, 3, 9, 82, 10};
33      mergeSort(arr, 0, arr.length - 1);
34      for (int x : arr) System.out.print(x + " ");
35      // Output: 3 9 10 27 38 43 82
36  }
37  }

```

Listing 7.4: Merge Sort — stable $O(n \log n)$ guaranteed

Dry Run: arr = {38, 27, 43, 3}

**Complexity Analysis**

Time: $O(n \log n)$ in all cases (best, average, worst) Space: $O(n)$ auxiliary Stable: Yes
 Preferred for sorting **Linked Lists**. Best choice when stability is required on large data.

7.5 Quick Sort

Idea: Choose a **pivot** element. Partition the array so all elements $<$ pivot go left and all $>$ pivot go right. Recursively sort both sides.

```

1  public class QuickSort {
2      // Partition: place pivot at its correct sorted position
3      static int partition(int[] arr, int low, int high) {
4          int pivot = arr[high]; // choose last element as pivot

```

```

5      int i      = low - 1;      // i tracks boundary of smaller elements
6
7      for (int j = low; j < high; j++) {
8          if (arr[j] <= pivot) {
9              i++;
10             // Swap arr[i] and arr[j]
11             int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
12         }
13     }
14     // Place pivot in its correct position
15     int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
16     return i + 1;    // return pivot's final index
17 }
18
19 static void quickSort(int[] arr, int low, int high) {
20     if (low < high) {
21         int pivotIndex = partition(arr, low, high); // pivot is placed
22             correctly
23         quickSort(arr, low, pivotIndex - 1);        // sort left of pivot
24         quickSort(arr, pivotIndex + 1, high);        // sort right of
25             pivot
26     }
27 }
28
29 public static void main(String[] args) {
30     int[] arr = {10, 7, 8, 9, 1, 5};
31     quickSort(arr, 0, arr.length - 1);
32     for (int x : arr) System.out.print(x + " ");
33     // Output: 1 5 7 8 9 10
34 }

```

Listing 7.5: Quick Sort — in-place, average $O(n \log n)$ **Dry Run: Partition on {10, 7, 8, 9, 1, 5}, pivot = 5**

j	arr[j]	arr[j] ≤ 5?	i	Action
0	10	No	-1	skip
1	7	No	-1	skip
2	8	No	-1	skip
3	9	No	-1	skip
4	1	Yes	0	swap arr[0] ↔ arr[4] → {1,7,8,9,10,5}
End of loop: swap arr[i+1]=arr[1] with pivot arr[5]=5 → {1,5,8,9,10,7}				
Pivot 5 is at index 1. Left {1}, Right {8,9,10,7} sorted recursively.				

Complexity Analysis

Best/Average: $O(n \log n)$ Worst: $O(n^2)$ (sorted array with last-element pivot) Space: $O(\log n)$ call stack Stable: No

In practice, Quick Sort is the fastest in-memory sort due to excellent cache performance. Use randomised pivot to avoid $O(n^2)$ worst case.

Interview Corner

- “Why is Quick Sort preferred over Merge Sort in practice?” — Better cache locality; in-place ($O(1)$ extra space vs $O(n)$)
- “What is the worst case?” — Already sorted / reverse sorted array with fixed pivot
- “How do you avoid worst case?” — Random pivot selection; or use median-of-three
- Quick Sort is **not stable**. If stability needed, use Merge Sort

7.6 Heap Sort

Idea: Build a **Max-Heap** from the array. The largest element is always at the root (index 0). Swap it to the end, shrink the heap by one, and *heapify* to restore the heap property. Repeat until sorted.

Key concept — heapify: Given a node at index i in an array of size n , its children are at $2*i+1$ (left) and $2*i+2$ (right). Heapify pushes `arr[i]` down until it is larger than both children.

```

1 public class HeapSort {
2
3     // heapify: fix the heap rooted at index i, heap size = n
4     static void heapify(int[] arr, int n, int i) {
5         int largest = i;           // assume root is largest
6         int left    = 2 * i + 1;  // left child index
7         int right   = 2 * i + 2;  // right child index
8
9         // if left child exists and is larger than root
10        if (left < n && arr[left] > arr[largest])
11            largest = left;
12
13        // if right child exists and is larger than current largest
14        if (right < n && arr[right] > arr[largest])
15            largest = right;
16
17        // if root is not the largest, swap and continue heapifying
18        if (largest != i) {
19            int temp    = arr[i];
20            arr[i]      = arr[largest];
21            arr[largest] = temp;
22
23            heapify(arr, n, largest); // fix the affected subtree
24        }
25    }
26 }

```

```

25     }
26
27     static void heapSort(int[] arr) {
28         int n = arr.length;
29
30         // Step 1: Build max-heap from the array
31         // Start from last non-leaf node and heapify each node upward
32         for (int i = n / 2 - 1; i >= 0; i--)
33             heapify(arr, n, i);
34
35         // Step 2: Extract elements one by one from heap
36         for (int i = n - 1; i > 0; i--) {
37             // Move current root (largest) to end
38             int temp = arr[0];
39             arr[0] = arr[i];
40             arr[i] = temp;
41
42             // Heapify the reduced heap (ignore sorted tail)
43             heapify(arr, i, 0);
44         }
45     }
46
47     public static void main(String[] args) {
48         int[] arr = {12, 11, 13, 5, 6, 7};
49         heapSort(arr);
50         for (int x : arr) System.out.print(x + " ");
51         // Output: 5 6 7 11 12 13
52     }
53 }

```

Listing 7.6: Heap Sort — in-place $O(n \log n)$, not stable

Dry Run: arr = {4, 10, 3, 5, 1}

Step 1 — Build Max-Heap (heapify from index 1 down to 0):

Heapify at i	Array after
$i = 1$	{4, 10, 3, 5, 1} (10 > 4, swap → no, 10 already largest child)
$i = 0$	{10, 5, 3, 4, 1} (10 > 4,5; swap root with 10; then fix subtree)

Step 2 — Extract max repeatedly:

Pass	Swap (root $\leftrightarrow$ end)	Array	Sorted tail
1	arr[0] $\leftrightarrow$ arr[4] heapify(4,0)	{1,5,3,4,10} {5,4,3,1,10}	10
2	arr[0] $\leftrightarrow$ arr[3] heapify(3,0)	{1,4,3,5,10} {4,1,3,5,10}	5,10
3	arr[0] $\leftrightarrow$ arr[2] heapify(2,0)	{3,1,4,5,10} {3,1,4,5,10}	4,5,10
4	arr[0] $\leftrightarrow$ arr[1]	{1,3,4,5,10}	3,4,5,10
Result: {1, 3, 4, 5, 10}			

Complexity Analysis

Time: $O(n \log n)$ best, average and worst Space: $O(1)$ — fully in-place Stable: No
Best when memory is constrained. The $O(\log n)$ heapify runs n times for build and n times for extraction.

Interview Corner

- **Why not used in practice despite $O(n \log n)$?** — Poor cache locality; heap accesses jump around in memory, unlike Quick Sort which accesses nearby indices
- **When is it preferred?** — When guaranteed $O(n \log n)$ AND $O(1)$ space are both required
- **Is Heap Sort stable?** — No. Equal elements may swap during heapify
- **What is a heap?** — A complete binary tree stored as an array where every parent $\geq$ both children (max-heap)

7.7 Radix Sort

Idea: Sort integers digit by digit, from the **least significant digit (LSD)** to the most significant digit. Use a **stable sort** (Counting Sort) at each digit pass. Since each pass is $O(n)$, and there are d digit passes, total time is $O(d \cdot n)$.

Key insight: Radix Sort does *not* compare elements. It distributes them into buckets 0–9 based on the current digit, then collects them in order.

```

1 public class RadixSort {
2
3     // Counting Sort on a specific digit position (exp = 1, 10, 100, ...)
4     static void countingSort(int[] arr, int exp) {
5         int n = arr.length;

```

```

6      int[] output = new int[n]; // sorted output for this pass
7      int[] count  = new int[10]; // buckets 0 to 9
8
9      // Count how many numbers have digit d at position exp
10     for (int i = 0; i < n; i++) {
11         int digit = (arr[i] / exp) % 10;
12         count[digit]++;
13     }
14
15     // Convert count[] to cumulative: count[i] now holds
16     // the position of digit i in the output
17     for (int i = 1; i < 10; i++)
18         count[i] += count[i - 1];
19
20     // Build output array (traverse right to left for stability)
21     for (int i = n - 1; i >= 0; i--) {
22         int digit = (arr[i] / exp) % 10;
23         output[count[digit] - 1] = arr[i];
24         count[digit]--;
25     }
26
27     // Copy output back into arr
28     for (int i = 0; i < n; i++)
29         arr[i] = output[i];
30 }
31
32 static void radixSort(int[] arr) {
33     // Find maximum element to know number of digits
34     int max = arr[0];
35     for (int x : arr) if (x > max) max = x;
36
37     // Do one counting sort pass for each digit position
38     // exp = 1 (units), 10 (tens), 100 (hundreds), ...
39     for (int exp = 1; max / exp > 0; exp *= 10)
40         countingSort(arr, exp);
41 }
42
43 public static void main(String[] args) {
44     int[] arr = {170, 45, 75, 90, 802, 24, 2, 66};
45     radixSort(arr);
46     for (int x : arr) System.out.print(x + " ");
47     // Output: 2 24 45 66 75 90 170 802
48 }
49 }

```

Listing 7.7: Radix Sort — $O(d \cdot n)$ time, $O(n+b)$ space, stable

Dry Run: arr = {170, 45, 75, 90, 802, 24, 2, 66}

Max = 802 → 3 digit passes needed.

Pass	Sort by digit	Array after pass
1	Units (exp=1)	{170, 90, 802, 2, 24, 45, 75, 66}
2	Tens (exp=10)	{802, 2, 24, 45, 66, 170, 75, 90}
3	Hundreds (exp=100)	{2, 24, 45, 66, 75, 90, 170, 802}

Pass 1 detail (sort by units digit): Numbers grouped by last digit:

bucket[0]: 170, 90 bucket[2]: 802, 2 bucket[4]: 24 bucket[5]: 45 bucket[5]: 75
 bucket[6]: 66

Collect in order → {170, 90, 802, 2, 24, 45, 75, 66}

Complexity Analysis

Time: $O(d \times n)$ where d = number of digits in max element Space: $O(n + b)$ where b = base (10) Stable: Yes

For fixed-width integers (e.g. 32-bit), d is constant, making it effectively $O(n)$ — faster than any comparison sort.

Interview Corner

- **Why can Radix Sort beat $O(n \log n)$?** — It does not compare elements; it counts and places. Comparison sorts have a theoretical lower bound of $O(n \log n)$; non-comparison sorts can bypass this
- **When to use?** — Large arrays of integers with a bounded number of digits (phone numbers, IDs, scores)
- **Can it sort negative numbers?** — Not directly. Separate negatives, sort absolute values, reverse and negate, then merge with positives
- **Why traverse right to left in counting sort?** — To maintain stability: elements with the same digit keep their original relative order

7.8 Counting Sort

Idea: Instead of comparing elements, **count how many times each value appears**. Then use those counts to place every element directly at its correct position. Works only when the range of values k is known and reasonably small.

When to use: Elements are non-negative integers in a known range $[0, k]$. Example: sort exam scores (0–100), sort ages (0–120), sort characters.

```

1 public class CountingSort {
2
3     static void countingSort(int[] arr) {
4         int n = arr.length;
5     }

```

```

6      // Step 1: Find the maximum value to know range of counts
7      int max = arr[0];
8      for (int x : arr) if (x > max) max = x;
9
10     // Step 2: Create count array of size max+1, initialised to 0
11     int[] count = new int[max + 1];
12
13     // Step 3: Count occurrences of each element
14     for (int x : arr)
15         count[x]++;
16     // count[i] = how many times value i appears in arr
17
18     // Step 4: Convert to cumulative counts
19     // count[i] now = number of elements <= i
20     for (int i = 1; i <= max; i++)
21         count[i] += count[i - 1];
22
23     // Step 5: Build output array (traverse right to left for stability)
24     int[] output = new int[n];
25     for (int i = n - 1; i >= 0; i--) {
26         output[count[arr[i]] - 1] = arr[i]; // place at correct position
27         count[arr[i]]--;                     // decrement for duplicates
28     }
29
30     // Step 6: Copy output back to arr
31     for (int i = 0; i < n; i++)
32         arr[i] = output[i];
33 }
34
35 public static void main(String[] args) {
36     int[] arr = {4, 2, 2, 8, 3, 3, 1};
37     countingSort(arr);
38     for (int x : arr) System.out.print(x + " ");
39     // Output: 1 2 2 3 3 4 8
40 }
41 }

```

Listing 7.8: Counting Sort — $O(n + k)$ time and space, stable

Dry Run: arr = {4, 2, 2, 8, 3, 3, 1}, max = 8

Step 3 — Count occurrences:

Index (value)	0	1	2	3	4	5	6	7	8
count[]	0	1	2	2	1	0	0	0	1

Step 4 — Cumulative counts:

Index	0	1	2	3	4	5	6	7	8
count[]	0	1	3	5	6	6	6	6	7

Step 5 — Place elements (right to left, `arr = {4, 2, 2, 8, 3, 3, 1}`):

i	arr[i]	count[arr[i]]	Placed at index
6	1	1	output[0] = 1
5	3	5	output[4] = 3
4	3	4	output[3] = 3
3	8	7	output[6] = 8
2	2	3	output[2] = 2
1	2	2	output[1] = 2
0	4	6	output[5] = 4
Result: {1, 2, 2, 3, 3, 4, 8}			

Complexity Analysis

Time: $O(n + k)$ where k = range of values (max element) Space: $O(n + k)$ for count and output arrays Stable: Yes

When $k = O(n)$, this is effectively $O(n)$ — the fastest possible sort. But if $k \gg n$ (e.g. sorting 10 numbers in range 0–10 000 000), it wastes huge memory.

Interview Corner

- **Why is it $O(n + k)$ and not $O(n)$?** — Even if the array has 5 elements but max is 1000, you still initialise and scan 1001 buckets
- **Can it handle negative numbers?** — Yes, with an offset. Find `min`, shift all values by `-min` so the range starts at 0, sort, then shift back
- **Can it handle floats?** — No. Only works for integer keys
- **Relationship to Radix Sort?** — Radix Sort uses Counting Sort internally as its per-digit stable sort. Understanding Counting Sort is the key to understanding Radix Sort
- **Why right-to-left in Step 5?** — Ensures stability: duplicate values appear in their original relative order in the output

7.9 Tim Sort

Idea: Tim Sort is a **hybrid sorting algorithm** that combines **Insertion Sort** (for small sub-arrays) and **Merge Sort** (for merging). It was designed by Tim Peters in 2002 for Python and is now used in Java's `Arrays.sort()` for objects and Python's built-in `sort()`.

Key concept — run: Tim Sort divides the array into small chunks called *runs* (typically 32–64 elements). Each run is sorted with Insertion Sort (which is extremely fast on small arrays).

Sorted runs are then merged using a merge similar to Merge Sort.

[

Why Tim Sort?] Real-world data is **rarely random**. It often has naturally ordered subsequences (partially sorted regions). Tim Sort detects these natural runs and exploits them, making it much faster than pure Merge Sort or Quick Sort on real data.

```

1  public class TimSort {
2
3      static final int RUN = 32; // size of each small run
4
5      // Step 1: Insertion Sort for a small subarray arr[left..right]
6      static void insertionSort(int[] arr, int left, int right) {
7          for (int i = left + 1; i <= right; i++) {
8              int key = arr[i];
9              int j = i - 1;
10             while (j >= left && arr[j] > key) {
11                 arr[j + 1] = arr[j];
12                 j--;
13             }
14             arr[j + 1] = key;
15         }
16     }
17
18     // Step 2: Merge two sorted runs arr[left..mid] and arr[mid+1..right]
19     static void merge(int[] arr, int left, int mid, int right) {
20         int len1 = mid - left + 1;
21         int len2 = right - mid;
22
23         int[] L = new int[len1];
24         int[] R = new int[len2];
25
26         for (int i = 0; i < len1; i++) L[i] = arr[left + i];
27         for (int i = 0; i < len2; i++) R[i] = arr[mid + 1 + i];
28
29         int i = 0, j = 0, k = left;
30         while (i < len1 && j < len2) {
31             if (L[i] <= R[j]) arr[k++] = L[i++]; // <= keeps it stable
32             else arr[k++] = R[j++];
33         }
34         while (i < len1) arr[k++] = L[i++];
35         while (j < len2) arr[k++] = R[j++];
36     }
37
38     // Step 3: Full Tim Sort
39     static void timSort(int[] arr) {
40         int n = arr.length;
41
42         // Sort each run of size RUN using Insertion Sort
43         for (int i = 0; i < n; i += RUN)
44             insertionSort(arr, i, Math.min(i + RUN - 1, n - 1));

```



```

45
46 // Merge runs: start with size RUN, then 2*RUN, 4*RUN, ...
47 for (int size = RUN; size < n; size *= 2) {
48     for (int left = 0; left < n; left += 2 * size) {
49         int mid = Math.min(left + size - 1, n - 1);
50         int right = Math.min(left + 2 * size - 1, n - 1);
51
52         if (mid < right) // only merge if there is a right run
53             merge(arr, left, mid, right);
54     }
55 }
56
57
58 public static void main(String[] args) {
59     int[] arr = {5, 21, 7, 23, 19, 2, 11, 3, 8, 14};
60     timSort(arr);
61     for (int x : arr) System.out.print(x + " ");
62     // Output: 2 3 5 7 8 11 14 19 21 23
63 }
64 }

```

Listing 7.9: Tim Sort — simplified implementation showing core idea

Dry Run concept: arr = {5, 21, 7, 23, 19, 2, 11, 3, 8, 14}, RUN = 4

Phase	What happens
Split into runs	Run 1: {5,21,7,23} Run 2: {19,2,11,3} Run 3: {8,14}
Insertion Sort each run	Run 1 → {5,7,21,23} Run 2 → {2,3,11,19} Run 3 → {8,14} (already sorted)
Merge pass 1 (size=4)	Merge Run1 + Run2 → {2,3,5,7,11,19,21,23}
Merge pass 2 (size=8)	Merge result + Run3 → {2,3,5,7,8,11,14,19,21,23}

Complexity Analysis

Time: $O(n \log n)$ worst case Best: $O(n)$ when array is already sorted (natural runs cover everything) Space: $O(n)$ for merge Stable: Yes

In practice, Tim Sort outperforms pure Merge Sort and Quick Sort on real-world data because it exploits existing order. This is why Java and Python use it as their default sort.

Interview Corner

- **“What does `Arrays.sort()` use in Java?”** — For primitive arrays (`int[]`, `double[]`) Java uses **Dual-Pivot Quick Sort**. For object arrays (`Integer[]`, `String[]`) Java uses **Tim Sort** because object sorting requires stability
- **Why Insertion Sort for small runs?** — For $n \leq 32$, Insertion Sort’s low overhead beats Merge Sort’s recursive splitting cost. This crossover point is well-studied
- **What is a “natural run”?** — A maximal already-ascending (or descending) subsequence. Tim Sort detects these and avoids re-sorting them
- **Why is Tim Sort stable?** — The $\leq$ in the merge step ensures equal elements from the left run come before those from the right run, preserving original order
- **Tim Sort vs Merge Sort?** — Tim Sort is Merge Sort with two key optimisations: uses Insertion Sort for small chunks, and exploits pre-existing order in real data

7.10 Sorting Algorithms — When to Use Which

Situation	Best Choice	Why
Small array ($n \leq 20$)	Insertion Sort	Low overhead; nearly $O(n)$ if nearly sorted
General purpose, in-place	Quick Sort	Best average cache performance
Stability required	Merge Sort	Stable; $O(n \log n)$ guaranteed
Guaranteed $O(n \log n)$ + $O(1)$ space	Heap Sort	No $O(n^2)$ worst case, no extra memory
Large integer arrays	Radix Sort	Effectively $O(n)$ for bounded digit count
Small range integers	Counting Sort	$O(n + k)$ — fastest when range k is small
Real-world Java / Python sort	Tim Sort	What <code>Arrays.sort()</code> actually uses internally
Linked List sorting	Merge Sort	No random access needed
Nearly sorted data	Insertion Sort	Best case $O(n)$

ArrayList

An **ArrayList** is a resizable array from `java.util`. Unlike a plain array, its size grows and shrinks automatically. Internally it uses an array; when that array is full, Java creates a new one roughly 1.5× larger and copies everything across.

ArrayList vs Array

Feature	Array	ArrayList
Size	Fixed at creation	Dynamic (grows/shrinks)
Type	Primitives allowed	Objects only (wrappers)
Syntax	<code>int[] a</code>	<code>ArrayList<Integer></code>
Import	None	<code>java.util.ArrayList</code>
Length	<code>a.length</code>	<code>a.size()</code>
Access	<code>a[i]</code>	<code>a.get(i)</code>
Set	<code>a[i] = v</code>	<code>a.set(i, v)</code>
Add element	Not possible	<code>a.add(v)</code>
Remove	Not possible	<code>a.remove(i)</code>

8.1 Creating an ArrayList

```

1 import java.util.ArrayList;
2
3 public class ArrayListBasics {
4     public static void main(String[] args) {
5
6         // 1. Empty ArrayList of integers
7         ArrayList<Integer> nums = new ArrayList<>();
8
9         // 2. ArrayList with initial capacity hint (does NOT set size)

```

```

10     ArrayList<Integer> big = new ArrayList<>(100);
11
12     // 3. Pre-filled using add()
13     ArrayList<String> names = new ArrayList<>();
14     names.add("Alice");
15     names.add("Bob");
16     names.add("Charlie");
17
18     System.out.println(names);           // [Alice, Bob, Charlie]
19     System.out.println(names.size());    // 3
20 }
21 }

```

Listing 8.1: ArrayList — Declaration and Initialisation

Wrapper Classes

ArrayList cannot store primitives directly. Use wrapper classes:

int → Integer, double → Double, char → Character, boolean → Boolean
 Java auto-boxes and unboxes automatically: `nums.add(5)` works even though 5 is a primitive.

8.2 Common Operations

```

1  import java.util.ArrayList;
2  import java.util.Collections;
3
4  public class ArrayListOps {
5      public static void main(String[] args) {
6
7          ArrayList<Integer> list = new ArrayList<>();
8
9          // ADD
10         list.add(10);           // append to end      -> [10]
11         list.add(20);           //                    -> [10, 20]
12         list.add(30);           //                    -> [10, 20, 30]
13         list.add(1, 99);        // insert at index 1  -> [10, 99, 20, 30]
14
15         // ACCESS
16         int first = list.get(0); // 10
17         int last  = list.get(list.size() - 1); // 30
18
19         // UPDATE
20         list.set(1, 55);         // replace index 1   -> [10, 55, 20, 30]
21
22         // REMOVE
23         list.remove(2);          // remove by index   -> [10, 55, 30]
24         list.remove(Integer.valueOf(55)); // remove by value -> [10, 30]
25     }

```

```
26      // SEARCH
27      boolean has = list.contains(30); // true
28      int idx     = list.indexOf(30);  // 1
29
30      // SIZE & EMPTY
31      int sz      = list.size();        // 2
32      boolean e   = list.isEmpty();    // false
33
34      // SORT & REVERSE
35      list.add(5); list.add(40); list.add(1);
36      Collections.sort(list);           // [1, 5, 10, 30, 40]
37      Collections.reverse(list);        // [40, 30, 10, 5, 1]
38      Collections.sort(list, Collections.reverseOrder()); // descending
39
40      // CLEAR
41      list.clear();                      // []
42      System.out.println(list.isEmpty()); // true
43  }
44 }
```

Listing 8.2: ArrayList — All Core Operations

8.3 Traversal Methods

```
1  import java.util.ArrayList;
2
3  public class Traverse {
4      public static void main(String[] args) {
5
6          ArrayList<Integer> list = new ArrayList<>();
7          for (int i = 1; i <= 5; i++) list.add(i * 10);
8          // list = [10, 20, 30, 40, 50]
9
10         // 1. Traditional for loop (when index needed)
11         for (int i = 0; i < list.size(); i++) {
12             System.out.print(list.get(i) + " ");
13         }
14
15         // 2. Enhanced for-each (simplest, read-only)
16         for (int x : list) {
17             System.out.print(x + " ");
18         }
19
20         // 3. While loop
21         int i = 0;
22         while (i < list.size()) {
23             System.out.print(list.get(i) + " ");
24             i++;
25         }
26     }
```

```

27     // 4. forEach with lambda (Java 8+)
28     list.forEach(x -> System.out.print(x + " "));
29 }
30 }

```

Listing 8.3: ArrayList — Four Ways to Traverse

8.4 Size of an ArrayList

Problem: Print the size of an ArrayList before and after adding elements.

```

1  import java.util.ArrayList;
2
3  public class SizeDemo {
4      public static void main(String[] args) {
5          ArrayList<Integer> list = new ArrayList<>();
6          System.out.println("Before: " + list.size()); // 0
7
8          list.add(5);
9          list.add(10);
10         list.add(15);
11
12         System.out.println("After:  " + list.size()); // 3
13
14         list.remove(0);
15         System.out.println("After remove: " + list.size()); // 2
16     }
17 }

```

Listing 8.4: ArrayList — size() demo

list starts empty

Operation	list state	size()
initial	[]	0
add(5)	[5]	1
add(10)	[5, 10]	2
add(15)	[5, 10, 15]	3
remove(0)	[10, 15]	2

8.5 Print Reverse of an ArrayList

Problem: Print all elements of an ArrayList in reverse order without modifying it.

```

1  import java.util.ArrayList;
2
3  public class PrintReverse {
4      public static void printReverse(ArrayList<Integer> list) {
5          // Start from last index, go down to 0
6          for (int i = list.size() - 1; i >= 0; i--) {
7              System.out.print(list.get(i) + " ");
8          }
9          System.out.println();
10     }
11
12     public static void main(String[] args) {
13         ArrayList<Integer> list = new ArrayList<>();
14         list.add(10); list.add(20); list.add(30); list.add(40); list.add(50);
15         // Original: [10, 20, 30, 40, 50]
16         printReverse(list); // Output: 50 40 30 20 10
17     }
18 }

```

Listing 8.5: Print Reverse — $O(n)$ time $O(1)$ extra space

list = [10, 20, 30, 40, 50]

i	list.get(i)	printed
4	50	50
3	40	40
2	30	30
1	20	20
0	10	10

Complexity Analysis

Time: $O(n)$ — one reverse pass Space: $O(1)$ extra (no copy made)

8.6 Swap Two Numbers in an ArrayList

Problem: Swap elements at two given indices in an ArrayList.

```

1  import java.util.ArrayList;
2  import java.util.Collections;

```

```

3
4 public class SwapNumbers {
5     public static void main(String[] args) {
6         ArrayList<Integer> list = new ArrayList<>();
7         list.add(10); list.add(20); list.add(30); list.add(40);
8
9         int i = 1, j = 3; // swap index 1 and index 3
10
11        // Method 1: Manual swap using set()
12        int temp = list.get(i);
13        list.set(i, list.get(j));
14        list.set(j, temp);
15        System.out.println(list); // [10, 40, 30, 20]
16
17        // Method 2: Using Collections.swap() -- one-liner
18        Collections.swap(list, 0, 2);
19        System.out.println(list); // [30, 40, 10, 20]
20    }
21 }

```

Listing 8.6: Swap Two Numbers in ArrayList

list = [10, 20, 30, 40], swap i=1, j=3

Step	Action	list
Initial	—	[10, 20, 30, 40]
temp=20	temp = get(1)	[10, 20, 30, 40]
set(1,40)	copy get(3) to index 1	[10, 40, 30, 40]
set(3,20)	copy temp to index 3	[10, 40, 30, 20]

Complexity Analysis

Time: $O(1)$ Space: $O(1)$

8.7 Find Maximum in an ArrayList

Problem: Find the largest element in an ArrayList.

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3
4 public class FindMax {
5     public static int findMax(ArrayList<Integer> list) {
6         int max = list.get(0); // assume first is max
7         for (int i = 1; i < list.size(); i++) {

```



```

8         if (list.get(i) > max) {
9             max = list.get(i);    // update whenever we find a bigger value
10        }
11    }
12    return max;
13 }
14
15 public static void main(String[] args) {
16     ArrayList<Integer> list = new ArrayList<>();
17     list.add(3); list.add(7); list.add(1); list.add(9); list.add(4);
18
19     System.out.println("Max: " + findMax(list));    // 9
20     System.out.println("Max (API): " + Collections.max(list)); // 9
21 }
22 }

```

Listing 8.7: Find Maximum — Linear Scan

list = [3, 7, 1, 9, 4]

i	list.get(i)	max
—	—	3 (initial)
1	7	7
2	1	7
3	9	9
4	4	9

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

8.8 Sorting an ArrayList

Problem: Sort an ArrayList in ascending and descending order.

```

1 import java.util.ArrayList;
2 import java.util.Collections;
3
4 public class SortArrayList {
5     public static void main(String[] args) {
6         ArrayList<Integer> list = new ArrayList<>();
7         list.add(5); list.add(1); list.add(9); list.add(3); list.add(7);
8
9         // Ascending (natural order)

```

```
10 Collections.sort(list);
11 System.out.println("Ascending: " + list); // [1, 3, 5, 7, 9]
12
13 // Descending (reverse order)
14 Collections.sort(list, Collections.reverseOrder());
15 System.out.println("Descending: " + list); // [9, 7, 5, 3, 1]
16 }
17 }
```

Listing 8.8: Sort ArrayList — Ascending and Descending

Sorting Strings

`Collections.sort()` also works on `ArrayList<String>` — it sorts lexicographically (dictionary order).

Complexity Analysis

Time: $O(n \log n)$ — Tim Sort internally Space: $O(n)$

8.9 Multi-Dimensional ArrayList

A 2-D ArrayList is an ArrayList of ArrayLists. Each inner list can have a different length, making it a **jagged** structure — unlike a 2-D array which is always rectangular.

```
1 import java.util.ArrayList;
2
3 public class MultiDim {
4     public static void main(String[] args) {
5
6         // A 2-D ArrayList (list of lists)
7         ArrayList<ArrayList<Integer>> matrix = new ArrayList<>();
8
9         // Add three rows of different lengths
10        ArrayList<Integer> row0 = new ArrayList<>();
11        row0.add(1); row0.add(2); row0.add(3);
12
13        ArrayList<Integer> row1 = new ArrayList<>();
14        row1.add(4); row1.add(5);
15
16        ArrayList<Integer> row2 = new ArrayList<>();
17        row2.add(6); row2.add(7); row2.add(8); row2.add(9);
18
19        matrix.add(row0);
20        matrix.add(row1);
21        matrix.add(row2);
22
23        // Print
```

```
24     for (int i = 0; i < matrix.size(); i++) {
25         for (int j = 0; j < matrix.get(i).size(); j++) {
26             System.out.print(matrix.get(i).get(j) + " ");
27         }
28         System.out.println();
29     }
30     // Output:
31     // 1 2 3
32     // 4 5
33     // 6 7 8 9
34 }
35 }
```

Listing 8.9: 2-D ArrayList — Declare, Fill, and Print

row 0	1	2	3	
row 1	4	5		
row 2	6	7	8	9

When to Use 2-D ArrayList vs 2-D Array

Use `ArrayList<ArrayList<Integer>>` when rows have different lengths (Pascal's Triangle, adjacency list for graphs). Use `int[][]` when the grid is always rectangular and size is fixed.

8.10 ArrayList — Interview Corner

ArrayList — Must-Know for Interviews

- **Time complexities:** `add()` amortized $O(1)$; `get(i)` $O(1)$; `remove(i)` $O(n)$ due to shifting; `contains()` $O(n)$
- **ArrayList vs LinkedList:** ArrayList gives $O(1)$ random access; LinkedList gives $O(1)$ insert/delete at ends. Use ArrayList for most cases.
- **ArrayList vs Array:** When size is unknown at compile time, use ArrayList. When primitives and performance are critical, use array.
- **Auto-boxing cost:** Every `add(int)` boxes the `int` to `Integer` — negligible for small lists but relevant in hot loops.
- **ConcurrentModificationException:** Never add/remove inside a for-each loop. Use an explicit index loop or iterator's `remove()` instead.
- **Return type in problems:** Most LeetCode/interview problems return `List<Integer>` or `List<List<Integer>>` — ArrayList is the standard concrete implementation.
- **Convert array $\leftrightarrow$ ArrayList:**
 - Array to List: `Arrays.asList(arr)` (fixed size) or loop + `add()`
 - List to Array: `list.toArray(new Integer[0])`

Array Problems — Medium Level

9.1 Sort 0s, 1s, and 2s — Dutch National Flag

Problem: Sort an array containing only 0, 1, and 2, *in-place*, in one pass.

```

1  public class SortColors {
2      public static void sort012(int[] arr) {
3          int low  = 0;                // everything before low is 0
4          int mid  = 0;                // current element under inspection
5          int high = arr.length - 1;  // everything after high is 2
6
7          while (mid <= high) {
8              if (arr[mid] == 0) {
9                  // Swap arr[low] and arr[mid]; both 0-region and mid advance
10                 int t = arr[low]; arr[low] = arr[mid]; arr[mid] = t;
11                 low++;
12                 mid++;
13             } else if (arr[mid] == 1) {
14                 mid++;                // 1 is in the correct middle region; just move
                                     on
15             } else {                 // arr[mid] == 2
16                 // Swap arr[mid] and arr[high]; only high retreats
17                 int t = arr[mid]; arr[mid] = arr[high]; arr[high] = t;
18                 high--;
19                 // DO NOT increment mid: element swapped from high is unchecked
20                 !
21             }
22         }
23
24         public static void main(String[] args) {
25             int[] arr = {2, 0, 2, 1, 1, 0};
26             sort012(arr);
27             for (int x : arr) System.out.print(x + " ");
28             // Output: 0 0 1 1 2 2
29         }
30     }

```

Listing 9.1: Dutch National Flag Algorithm — $O(n)$ time $O(1)$ space

Three-region invariant maintained at all times:

All 0s [0 ...low-1]	All 1s [low ...mid-1]	Unknown [mid ...high]	All 2s [high+1 ...n-1]
------------------------	--------------------------	--------------------------	---------------------------

Dry Run: arr = {2, 0, 2, 1, 1, 0}

low	mid	high	arr[mid]	Array	Action
0	0	5	2	{2,0,2,1,1,0}	swap(mid,high), high-
0	0	4	0	{0,0,2,1,1,2}	swap(mid,low), low++,mid++
1	1	4	0	{0,0,2,1,1,2}	swap(mid,low), low++,mid++
2	2	4	2	{0,0,2,1,1,2}	swap(mid,high), high-
2	2	3	1	{0,0,1,1,2,2}	mid++
2	3	3	1	{0,0,1,1,2,2}	mid++
mid(4) > high(3): done. Result: {0,0,1,1,2,2}					

Complexity Analysis

Time: $O(n)$ — single pass Space: $O(1)$ — in-place

Interview Corner

Known as the “**Dutch National Flag Algorithm**”, invented by Edsger Dijkstra. Critical rule: when swapping arr[mid] with arr[high], do **not** increment mid — the element that came from arr[high] has never been inspected.

9.2 Majority Element — Boyer-Moore Voting

Problem: Find the element that appears **more than** $n/2$ times.

```

1 public class MajorityElement {
2     public static int majorityElement(int[] arr) {
3         int candidate = arr[0];
4         int count      = 1;
5
6         // Phase 1: Find the candidate using vote cancellation
7         for (int i = 1; i < arr.length; i++) {
8             if (arr[i] == candidate) {

```

```

9         count++; // same element: vote for it
10     } else {
11         count--; // different: cancel one vote
12         if (count == 0) {
13             candidate = arr[i]; // candidate eliminated; pick new one
14             count = 1;
15         }
16     }
17 }
18
19 // Phase 2: Verify (necessary when majority not guaranteed to exist)
20 int frequency = 0;
21 for (int x : arr) {
22     if (x == candidate) frequency++;
23 }
24 return (frequency > arr.length / 2) ? candidate : -1;
25 }
26
27 public static void main(String[] args) {
28     int[] arr = {3, 2, 3, 1, 3, 4, 3};
29     System.out.println(majorityElement(arr)); // 3
30 }
31 }

```

Listing 9.2: Boyer-Moore Voting Algorithm

Dry Run: arr = {3, 2, 3, 1, 3, 4, 3}

i	arr[i]	candidate / count	Note
0	3	3 / 1	Initialise
1	2	3 / 0	different: count- → 0
2	3	3 / 1	count=0, new candidate = 3
3	1	3 / 0	different: count- → 0
4	3	3 / 1	new candidate = 3
5	4	3 / 0	different: count- → 0
6	3	3 / 1	new candidate = 3

Verify: 3 appears 4 times; $4 > 7/2 = 3 \checkmark$ **Result:** 3

Complexity Analysis

Time: $O(n)$ two passes Space: $O(1)$ (HashMap approach: $O(n)$ time + $O(n)$ space)

9.3 Kadane's Algorithm — Maximum Subarray Sum

Problem: Find the contiguous subarray with the largest sum.

Example: $\text{arr} = \{-2, 1, -3, 4, -1, 2, 1, -5, 4\} \rightarrow \text{answer} = 6$ (subarray $\{4, -1, 2, 1\}$)

```

1  public class KadanesAlgorithm {
2      public static int maxSubarraySum(int[] arr) {
3          int maxSum = arr[0];    // best sum seen overall
4          int currentSum = arr[0]; // best sum ending at current index
5
6          for (int i = 1; i < arr.length; i++) {
7              // Either extend the previous subarray OR start a fresh one
8              currentSum = Math.max(arr[i], currentSum + arr[i]);
9              maxSum = Math.max(maxSum, currentSum);
10         }
11         return maxSum;
12     }
13
14     // BONUS: also return the subarray's start and end indices
15     public static void maxSubarrayWithIndices(int[] arr) {
16         int maxSum = arr[0], currentSum = arr[0];
17         int start = 0, end = 0, tempStart = 0;
18
19         for (int i = 1; i < arr.length; i++) {
20             if (arr[i] > currentSum + arr[i]) {
21                 currentSum = arr[i];    // start fresh subarray
22                 tempStart = i;
23             } else {
24                 currentSum += arr[i];    // extend current subarray
25             }
26             if (currentSum > maxSum) {
27                 maxSum = currentSum;
28                 start = tempStart;
29                 end = i;
30             }
31         }
32         System.out.println("Max Sum = " + maxSum);
33         System.out.print("Subarray: ");
34         for (int i = start; i <= end; i++) System.out.print(arr[i] + " ");
35     }
36
37     public static void main(String[] args) {
38         int[] arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4};
39         System.out.println(maxSubarraySum(arr));    // 6
40         maxSubarrayWithIndices(arr);                // Max Sum=6, Subarray: 4 -1
41                                                     2 1
42     }

```

Listing 9.3: Kadane's Algorithm — $O(n)$ time $O(1)$ space

Dry Run: `arr = {-2, 1, -3, 4, -1, 2, 1, -5, 4}`

i	arr[i]	currentSum	maxSum
init	—	-2	-2
1	1	$\max(1, -2 + 1) = 1$	1
2	-3	$\max(-3, 1 - 3) = -2$	1
3	4	$\max(4, -2 + 4) = 4$	4
4	-1	$\max(-1, 4 - 1) = 3$	4
5	2	$\max(2, 3 + 2) = 5$	5
6	1	$\max(1, 5 + 1) = 6$	6
7	-5	$\max(-5, 6 - 5) = 1$	6
8	4	$\max(4, 1 + 4) = 5$	6

Result: 6

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

Interview Corner

One of the most famous DP/greedy problems. The key insight: at each index, decide whether to **extend** the existing subarray or **start fresh**. If `currentSum` goes negative, starting fresh is always better.

9.4 Rearrange Array by Sign

Problem: Given an array with equal numbers of positive and negative integers, rearrange them alternately (positive first) without changing their relative order.

```

1 public class RearrangeBySign {
2     public static int[] rearrangeArray(int[] arr) {
3         int n      = arr.length;
4         int[] result = new int[n];
5         int posIdx = 0;    // next position for positive: 0, 2, 4, ...
6         int negIdx = 1;    // next position for negative: 1, 3, 5, ...
7
8         for (int i = 0; i < n; i++) {
9             if (arr[i] > 0) {
10                 result[posIdx] = arr[i];
11                 posIdx += 2;
12             } else {

```

```

13         result[negIdx] = arr[i];
14         negIdx += 2;
15     }
16 }
17 return result;
18 }
19
20 public static void main(String[] args) {
21     int[] arr = {3, 1, -2, -5, 2, -4};
22     int[] res = rearrangeArray(arr);
23     for (int x : res) System.out.print(x + " ");
24     // Output: 3 -2 1 -5 2 -4
25 }
26 }

```

Listing 9.4: Rearrange Alternating Positive and Negative

Dry Run: arr = {3, 1, -2, -5, 2, -4}

i	arr[i]	Sign	Placed at	result state
0	3	+ve	posIdx=0	{3,_,_,_,_,_}
1	1	+ve	posIdx=2	{3,_,1,_,_,_}
2	-2	-ve	negIdx=1	{3,-2,1,_,_,_}
3	-5	-ve	negIdx=3	{3,-2,1,-5,_,_}
4	2	+ve	posIdx=4	{3,-2,1,-5,2,_}
5	-4	-ve	negIdx=5	{3,-2,1,-5,2,-4}

Result: {3, -2, 1, -5, 2, -4}

Complexity Analysis

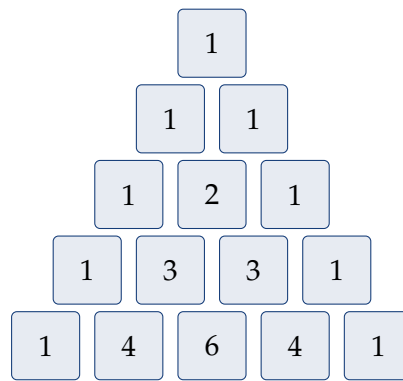
Time: $O(n)$ Space: $O(n)$ for the result array

When counts are unequal

The above assumes equal counts. If positive and negative counts differ, collect them separately into two lists, interleave up to the shorter count, then append the remainder.

9.5 Pascal's Triangle

Structure: Each element is the sum of the two elements directly above it. The boundary elements are always 1.



```

1 public class PascalsTriangle {
2     // Problem 1: Print entire Pascal's Triangle up to n rows
3     public static void printTriangle(int n) {
4         for (int row = 0; row < n; row++) {
5             int val = 1;
6             for (int col = 0; col <= row; col++) {
7                 System.out.print(val + " ");
8                 val = val * (row - col) / (col + 1); // next element in row
9             }
10            System.out.println();
11        }
12    }
13
14    // Problem 2: Return only the r-th row (0-indexed)
15    public static int[] getRow(int r) {
16        int[] row = new int[r + 1];
17        row[0] = 1;
18        for (int col = 1; col <= r; col++) {
19            // row[col] = row[col-1] * (r - col + 1) / col
20            row[col] = (int)((long) row[col - 1] * (r - col + 1) / col);
21        }
22        return row;
23    }
24
25    // Problem 3: Find element at row r, column c (both 0-indexed) = C(r, c)
26    public static long getElement(int r, int c) {
27        if (c > r - c) c = r - c; // use symmetry: C(r, c) = C(r, r-c)
28        long result = 1;
29        for (int i = 0; i < c; i++) {
30            result = result * (r - i) / (i + 1);
31        }
32        return result;
33    }
34
35    public static void main(String[] args) {
36        System.out.println("Pascal's Triangle (5 rows):");
37        printTriangle(5);
38
39        System.out.print("Row 4: ");
40        for (int x : getRow(4)) System.out.print(x + " "); // 1 4 6 4 1
41        System.out.println();
42    }

```

```

43     System.out.println("Element at row=4, col=2: " + getElement(4, 2)); //
        6
44     }
45 }

```

Listing 9.5: Print Pascal's Triangle — n rows

Dry Run: `getRow(4)` — row index 4 = {1, 4, 6, 4, 1}

col	row[col-1]	row[col] = prev × (r-col+1) / col	row
0	—	row[0] = 1	{1,_,_,_,_}
1	1	$1 \times (4 - 1 + 1) / 1 = 4$	{1,4,_,_,_}
2	4	$4 \times (4 - 2 + 1) / 2 = 6$	{1,4,6,_,_}
3	6	$6 \times (4 - 3 + 1) / 3 = 4$	{1,4,6,4,_}
4	4	$4 \times (4 - 4 + 1) / 4 = 1$	{1,4,6,4,1}

Complexity Analysis

Print n rows: $O(n^2)$ time, $O(n^2)$ space Get single row: $O(n)$ time, $O(n)$ space Get single element: $O(c)$ time, $O(1)$ space

9.6 3-Sum Problem

Problem: Find all unique triplets in the array that sum to zero.

```

1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.List;
4
5  public class ThreeSum {
6      public static List<List<Integer>> threeSum(int[] arr) {
7          List<List<Integer>> result = new ArrayList<>();
8          Arrays.sort(arr); // sort first -- enables two-pointer approach
9          int n = arr.length;
10
11         for (int i = 0; i < n - 2; i++) {
12             // Skip duplicate values for the fixed element
13             if (i > 0 && arr[i] == arr[i - 1]) continue;
14
15             int left = i + 1;
16             int right = n - 1;
17
18             while (left < right) {
19                 int sum = arr[i] + arr[left] + arr[right];

```

```

20
21         if (sum == 0) {
22             result.add(Arrays.asList(arr[i], arr[left], arr[right]));
23             // Skip duplicates for left and right pointers
24             while (left < right && arr[left] == arr[left + 1]) left
25                 ++;
26             while (left < right && arr[right] == arr[right - 1]) right
27                 --;
28             left++;
29             right--;
30         } else if (sum < 0) {
31             left++; // sum too small, increase left
32         } else {
33             right--; // sum too large, decrease right
34         }
35     }
36     return result;
37 }
38
39 public static void main(String[] args) {
40     int[] arr = {-1, 0, 1, 2, -1, -4};
41     System.out.println(threeSum(arr));
42     // Output: [[-1, -1, 2], [-1, 0, 1]]
43 }

```

Listing 9.6: 3-Sum — Sort + Two Pointer, $O(n^2)$

Dry Run: arr after sort = $\{-4, -1, -1, 0, 1, 2\}$

i	arr[i]	left	right	Action
0	-4	1	5	sum= -3 → left++
0	-4	2	5	sum= -3 → left++
0	-4	3	5	sum= -2 → left++
0	-4	4	5	sum= -1 → left++
0	-4	5	5	left > right → end
1	-1	2	5	sum= 0 Found! add[-1, -1, 2]
1	-1	3	4	sum= 0 Found! add[-1, 0, 1]
2	-1	—	—	arr[2]==arr[1]: skip duplicate

Result: $\{[-1, -1, 2], [-1, 0, 1]\}$

Complexity Analysis

Time: $O(n^2)$ — sort $O(n \log n)$ + two-pointer for each fixed element $O(n)$ Space: $O(1)$ extra (excluding result)

Interview Corner

- **Brute force** is $O(n^3)$ — three nested loops, check all triplets
- **HashSet** approach is also $O(n^2)$ but harder to deduplicate
- **Two-pointer after sort** is the standard interview solution
- **4-Sum**: Add one more outer loop, call 3-Sum logic inside $\rightarrow O(n^3)$
- Always skip duplicates after fixing each element to avoid duplicate triplets

9.7 Equilibrium Index

Problem: Find index i such that the sum of elements to its left equals the sum to its right.

```
1 public class EquilibriumIndex {
2     public static int equilibrium(int[] arr) {
3         int n = arr.length;
4
5         // Total sum of array
6         int totalSum = 0;
7         for (int x : arr) totalSum += x;
8
9         int leftSum = 0;
10        for (int i = 0; i < n; i++) {
11            // Right sum = totalSum - leftSum - arr[i]
12            int rightSum = totalSum - leftSum - arr[i];
13
14            if (leftSum == rightSum) return i; // equilibrium found
15
16            leftSum += arr[i]; // add current element to left sum
17        }
18        return -1; // no equilibrium index
19    }
20
21    public static void main(String[] args) {
22        int[] arr = {-7, 1, 5, 2, -4, 3, 0};
23        System.out.println(equilibrium(arr)); // 3 (arr[3]=2; left=-1, right
24        //=-1)
25    }
26 }
```

Listing 9.7: Equilibrium Index — Prefix Sum approach

Dry Run: `arr = {-7, 1, 5, 2, -4, 3, 0}`, `totalSum = 0`

i	arr[i]	leftSum	rightSum	Equal?
0	-7	0	$0 - 0 - (-7) = 7$	No
1	1	-7	$0 - (-7) - 1 = -6$	No
2	5	-6	$0 - (-6) - 5 = 1$	No
3	2	-1	$0 - (-1) - 2 = -1$	Yes! $-1 = -1$

Result: index 3

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

9.8 Missing Elements From an Array With Duplicates

Problem (LeetCode 448 / APNA CLG 489): Given an array `nums` of n integers where values are in the range $[1, n]$ and *some elements may be duplicated*, return **all integers** in $[1, n]$ that are missing from the array. The algorithm should run in $O(n)$ time and use $O(1)$ additional space.

Difference from “Find the Missing Number”

Section 5.16 (Find the Missing Number) handled the case where the array has *distinct* values and exactly *one* number is missing — solved in $O(n)$ with XOR or Gauss sum.

This problem allows *duplicates* and asks for *all* missing numbers — requires a different technique because Gauss sum and XOR no longer isolate individual missing values.

Approach — Negative Marking (Index as a Hash):

Since all values are in $[1, n]$ and indices are in $[0, n - 1]$, we can use the array itself as a hash table. For each value v encountered, mark `nums[v-1]` as negative (visited). Any index i whose value remains positive means $i + 1$ was never seen — it is missing.

```

1  import java.util.ArrayList;
2  import java.util.List;
3
4  public class MissingElementsWithDuplicates {
5
6      public static List<Integer> findMissingNumbers(int[] nums) {
7          int n = nums.length;
8          List<Integer> missing = new ArrayList<>();
9
10         // --- PASS 1: Mark visited positions as negative ---

```

```

11     for (int i = 0; i < n; i++) {
12         // Use absolute value because the number may already have been
           negated
13         int val = Math.abs(nums[i]);    // the value (1-indexed)
14         int idx = val - 1;              // the corresponding index (0-
           indexed)
15
16         if (nums[idx] > 0) {
17             nums[idx] = -nums[idx];    // mark as visited (negate)
18         }
19         // If nums[idx] is already negative: value was already seen (
           duplicate)
20     }
21
22     // --- PASS 2: Collect all indices whose value is still positive ---
23     for (int i = 0; i < n; i++) {
24         if (nums[i] > 0) {
25             missing.add(i + 1);        // index i means value (i+1) was
           never seen
26         }
27     }
28
29     // --- Optional PASS 3: Restore original array ---
30     for (int i = 0; i < n; i++) {
31         nums[i] = Math.abs(nums[i]);
32     }
33
34     return missing;
35 }
36
37 public static void main(String[] args) {
38     int[] arr1 = {4, 3, 2, 7, 8, 2, 3, 1};
39     System.out.println(findMissingNumbers(arr1));    // [5, 6]
40
41     int[] arr2 = {1, 1};
42     System.out.println(findMissingNumbers(arr2));    // [2]
43
44     int[] arr3 = {1, 2, 3};
45     System.out.println(findMissingNumbers(arr3));    // []
46 }
47 }

```

Listing 9.8: Missing Elements With Duplicates — Negative Marking, $O(n)$ time $O(1)$ space

nums = {4, 3, 2, 7, 8, 2, 3, 1}, n = 8

Pass 1 — Negative Marking:

i	nums[i]	val	idx	Array after marking
0	4	4	3	{4, 3, 2, -7, 8, 2, 3, 1}
1	3	3	2	{4, 3, -2, -7, 8, 2, 3, 1}
2	-2	2	1	{4, -3, -2, -7, 8, 2, 3, 1}
3	-7	7	6	{4, -3, -2, -7, 8, 2, -3, 1}
4	8	8	7	{4, -3, -2, -7, 8, 2, -3, -1}
5	2	2	1	already negative — skip
6	-3	3	2	already negative — skip
7	-1	1	0	{-4, -3, -2, -7, 8, 2, -3, -1}
Final: {-4,-3,-2,-7,8,2,-3,-1}				

Pass 2 — Collect missing:

Index 4 has value +8 > 0 → 5 is missing.

Index 5 has value +2 > 0 → 6 is missing.

All others are negative (their values were seen).

Result: [5, 6]

Original nums

4	3	2	7	8	2	3	1
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]

After Pass 1 (negatives = seen)

-4	-3	-2	-7	8	2	-3	-1
----	----	----	----	---	---	----	----

idx 4,5 still +ve → 5 and 6 missing

Complexity Analysis

Time: $O(n)$ — two passes over array Space: $O(1)$ extra (no auxiliary array; uses input array itself as hash)

Interview Corner

- **Why does negative marking work?** Values are in $[1, n]$ and indices in $[0, n - 1]$, so value v maps to index $v - 1$ bijectively. Negating `nums[v-1]` encodes “ v has been seen” without losing the original magnitude
- **Handling duplicates:** If `nums[idx]` is already negative, the value was already marked — we simply skip (no double-negation)
- **Always use `Math.abs()`** when reading during Pass 1, because earlier iterations may have already negated the value
- **Restore step** (Pass 3) is optional — skip if you don’t need the input array intact after the call
- Compared to a boolean visited array: same $O(n)$ time but $O(n)$ extra space — negative marking achieves $O(1)$ extra space by reusing the input

9.9 4-Sum Problem

Problem: Find all unique quadruplets in the array that sum to a given target.

Approach: Fix two elements with nested loops, then apply the Two-Pointer technique on the remaining sorted subarray — same idea as 3-Sum with one more outer loop.

```

1  import java.util.ArrayList;
2  import java.util.Arrays;
3  import java.util.List;
4
5  public class FourSum {
6      public static List<List<Integer>> fourSum(int[] arr, int target) {
7          List<List<Integer>> result = new ArrayList<>();
8          Arrays.sort(arr);           // sort first
9          int n = arr.length;
10
11         for (int i = 0; i < n - 3; i++) {
12             if (i > 0 && arr[i] == arr[i - 1]) continue; // skip i duplicates
13
14             for (int j = i + 1; j < n - 2; j++) {
15                 if (j > i + 1 && arr[j] == arr[j - 1]) continue; // skip j
16                                     duplicates
17
18                 int left = j + 1;
19                 int right = n - 1;
20
21                 while (left < right) {
22                     long sum = (long) arr[i] + arr[j] + arr[left] + arr[right];
23
24                     if (sum == target) {
25                         result.add(Arrays.asList(arr[i], arr[j], arr[left], arr
26                                     [right]));
27                         while (left < right && arr[left] == arr[left + 1])
28                             left++;

```

```

26         while (left < right && arr[right] == arr[right - 1])
27             right--;
28         left++;
29         right--;
30     } else if (sum < target) {
31         left++;
32     } else {
33         right--;
34     }
35 }
36 }
37 return result;
38 }
39
40 public static void main(String[] args) {
41     int[] arr = {1, 0, -1, 0, -2, 2};
42     System.out.println(fourSum(arr, 0));
43     // Output: [[-2,-1,1,2], [-2,0,0,2], [-1,0,0,1]]
44 }
45 }

```

Listing 9.9: 4-Sum — Fix Two + Two Pointer, $O(n^3)$

Complexity Analysis

Time: $O(n^3)$ — two fixed pointers + two-pointer pass Space: $O(1)$ extra (excluding output)

Interview Corner

- Use `long` for the sum to avoid integer overflow (4 ints can exceed `int` range)
- The deduplication logic (skip same values for `i`, `j`, `left`, `right`) is the tricky part
- Pattern generalises: k -Sum — fix $k - 2$ elements, do Two-Pointer on the rest: $O(n^{k-1})$

9.10 Next Permutation

Problem: Find the next lexicographically greater permutation of an array. If already the largest (fully descending), return the smallest (fully ascending).

Example: $[1, 2, 3] \rightarrow [1, 3, 2]$ $[3, 2, 1] \rightarrow [1, 2, 3]$ $[1, 1, 5] \rightarrow [1, 5, 1]$

Algorithm (3 steps, all $O(n)$):

1. Find the **pivot**: scan right to left, find first index i where `arr[i] < arr[i+1]`
2. Find the **successor**: scan right to left, find first element `> arr[i]`; swap with `arr[i]`
3. **Reverse** the suffix starting at $i+1$ (was descending; reversing gives ascending = smallest)

```
1 public class NextPermutation {
```

```

2   public static void nextPermutation(int[] arr) {
3       int n = arr.length;
4
5       // Step 1: Find pivot (first from right where arr[i] < arr[i+1])
6       int pivot = -1;
7       for (int i = n - 2; i >= 0; i--) {
8           if (arr[i] < arr[i + 1]) {
9               pivot = i;
10              break;
11          }
12      }
13
14      // No pivot means fully descending: reverse whole array
15      if (pivot == -1) {
16          reverse(arr, 0, n - 1);
17          return;
18      }
19
20      // Step 2: Find rightmost element > arr[pivot] and swap
21      for (int i = n - 1; i > pivot; i--) {
22          if (arr[i] > arr[pivot]) {
23              int temp = arr[pivot];
24              arr[pivot] = arr[i];
25              arr[i] = temp;
26              break;
27          }
28      }
29
30      // Step 3: Reverse suffix after pivot
31      reverse(arr, pivot + 1, n - 1);
32  }
33
34  private static void reverse(int[] arr, int start, int end) {
35      while (start < end) {
36          int temp = arr[start];
37          arr[start] = arr[end];
38          arr[end] = temp;
39          start++; end--;
40      }
41  }
42
43  public static void main(String[] args) {
44      int[] arr1 = {1, 2, 3}; nextPermutation(arr1);
45      for (int x : arr1) System.out.print(x + " "); // 1 3 2
46      System.out.println();
47
48      int[] arr2 = {3, 2, 1}; nextPermutation(arr2);
49      for (int x : arr2) System.out.print(x + " "); // 1 2 3
50      System.out.println();
51  }
52  }

```

Listing 9.10: Next Permutation — $O(n)$ time $O(1)$ space

Next Permutation of {2, 1, 5, 4, 3}

Step	Action	Array
Initial	—	{2,1,5,4,3}
Step 1	$\text{arr}[1]=1 < \text{arr}[2]=5 \rightarrow$ pivot=1	pivot at index 1
Step 2	rightmost >1 : swap[1]&[4]	$\text{arr}[4]=3$; {2,3,5,4,1}
Step 3	reverse suffix {5,4,1} $\rightarrow$ {1,4,5}	[2..4]: {2,3,1,4,5}

Result: {2, 3, 1, 4, 5}

Complexity Analysis

Time: $O(n)$ — three linear passes Space: $O(1)$ — in-place

Interview Corner

- This is exactly `std::next_permutation` from C++ STL
- **Why reverse the suffix?** After step 2, suffix is still descending. Reversing gives the smallest (ascending) arrangement
- Asked at Google, Facebook, Amazon, Microsoft
- Follow-up: *Previous permutation* — mirror logic: find first $\text{arr}[i] > \text{arr}[i+1]$, swap with rightmost smaller, reverse suffix

2-D Arrays (Matrices)

A **2-D array** (matrix) is an array of arrays — a table of rows and columns. It is the gateway to grid problems, image processing, graph adjacency matrices, and dynamic programming tables.

2-D Array

A 2-D array `int[][] mat` of size $m \times n$ has m rows and n columns. Element at row i , column j is accessed as `mat[i][j]` in $O(1)$.

10.1 Memory Representation

```
int[][] mat = new int[3][4]; // 3 rows, 4 columns
```

	Col 0	Col 1	Col 2	Col 3
Row 0	1	2	3	4
Row 1	5	6	7	8
Row 2	9	10	11	12

`[0][0]` `[0][1]` `[0][2]` `[0][3]`
`[1][0]` `[1][1]` `[1][2]` `[1][3]`
`[2][0]` `[2][1]` `[2][2]` `[2][3]`

Key Facts:

- `mat.length` → number of rows
- `mat[0].length` → number of columns
- Total elements: rows $\times$ columns
- Stored in **row-major order** in Java (each row is a separate 1-D array)
- Access `mat[i][j]`: $O(1)$ — direct index calculation

10.2 Declaring and Initialising a 2-D Array

```

1 public class MatrixBasics {
2     public static void main(String[] args) {
3
4         // Way 1: Declare size only (filled with 0s)
5         int[][] mat1 = new int[3][4];    // 3 rows, 4 columns
6
7         // Way 2: Declare with values (jagged is also allowed in Java)
8         int[][] mat2 = {
9             {1, 2, 3},
10            {4, 5, 6},
11            {7, 8, 9}
12        };
13
14        // Accessing a single element
15        System.out.println(mat2[1][2]);    // 6 (row 1, col 2)
16
17        // Length
18        System.out.println("Rows : " + mat2.length);    // 3
19        System.out.println("Cols : " + mat2[0].length);    // 3
20
21        // Traversal: nested for loop
22        System.out.println("Full matrix:");
23        for (int i = 0; i < mat2.length; i++) {
24            for (int j = 0; j < mat2[i].length; j++) {
25                System.out.print(mat2[i][j] + " ");
26            }
27            System.out.println();    // new row = new line
28        }
29    }
30 }

```

Listing 10.1: Declaring, initialising, and traversing a 2-D array

Traversal of 3×3 matrix $\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$

i (row)	j (col)	mat[i][j]	Output
0	0,1,2	1, 2, 3	1 2 3
1	0,1,2	4, 5, 6	4 5 6
2	0,1,2	7, 8, 9	7 8 9

Complexity Analysis

Traversal: Time $O(m \times n)$ Space $O(1)$

10.3 Print Diagonal Elements

Problem: Print all elements on the main diagonal of a square matrix. The main diagonal consists of elements where row index = column index: `mat[i][i]`.

1	2	3	← <code>mat[0][0]=1</code>
4	5	6	← <code>mat[1][1]=5</code>
7	8	9	← <code>mat[2][2]=9</code>

```
1 public class PrintDiagonal {
2     public static void printDiagonal(int[][] mat) {
3         // Only valid for square matrix (rows == cols)
4         for (int i = 0; i < mat.length; i++) {
5             System.out.print(mat[i][i] + " ");    // row index == col index
6         }
7         System.out.println();
8     }
9
10    public static void main(String[] args) {
11        int[][] mat = {
12            {1, 2, 3},
13            {4, 5, 6},
14            {7, 8, 9}
15        };
16        printDiagonal(mat);    // Output: 1 5 9
17    }
18 }
```

Listing 10.2: Print Diagonal Elements

Complexity Analysis

Time: $O(n)$ for an $n \times n$ matrix Space: $O(1)$

10.4 Sum of Diagonal Elements

```
1 public class DiagonalSum {
2     public static int diagonalSum(int[][] mat) {
3         int sum = 0;
4         for (int i = 0; i < mat.length; i++) {
5             sum += mat[i][i];    // accumulate diagonal elements
6         }
7     }
8 }
```



```

7     return sum;
8 }
9
10 public static void main(String[] args) {
11     int[][] mat = {
12         {1, 2, 3},
13         {4, 5, 6},
14         {7, 8, 9}
15     };
16     System.out.println("Diagonal sum = " + diagonalSum(mat)); // 15
17 }
18 }

```

Listing 10.3: Diagonal Sum — main diagonal only

```
arr = {{1,2,3},{4,5,6},{7,8,9}}
```

i	mat[i][i]	sum before	sum after
0	1	0	1
1	5	1	6
2	9	6	15

Result: 15

Both Diagonals Sum

For the **anti-diagonal** (top-right to bottom-left), the index is `mat[i][n-1-i]`. To sum both diagonals, add both — but if n is odd, subtract `mat[n/2][n/2]` once (counted twice in the middle).

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

10.5 Search Element in a Matrix

Problem: Given an $m \times n$ matrix, return `true` if target exists, else `false`.

10.5.1 Brute Force — Check All Elements

```

1 public class SearchMatrix {
2     public static boolean search(int[][] mat, int target) {
3         for (int i = 0; i < mat.length; i++) {
4             for (int j = 0; j < mat[0].length; j++) {

```

```
5         if (mat[i][j] == target) {
6             return true;    // found
7         }
8     }
9 }
10 return false;    // not found
11 }
12
13 public static void main(String[] args) {
14     int[][] mat = {
15         {1, 4, 7, 11},
16         {2, 5, 8, 12},
17         {3, 6, 9, 16},
18         {10, 13, 14, 17}
19     };
20     System.out.println(search(mat, 5));    // true
21     System.out.println(search(mat, 20));   // false
22 }
23 }
```

Listing 10.4: Search in Matrix — Brute Force $O(m \times n)$

Complexity Analysis

Time: $O(m \times n)$ Space: $O(1)$

10.5.2 Optimised — Staircase Search (Sorted Matrix)

If each row is sorted left-to-right and each column is sorted top-to-bottom, we can search in $O(m + n)$ by starting from the **top-right corner**.

```
1 public class SearchSortedMatrix {
2     public static boolean searchSorted(int[][] mat, int target) {
3         int row = 0;
4         int col = mat[0].length - 1;    // start at top-right corner
5
6         while (row < mat.length && col >= 0) {
7             if (mat[row][col] == target) {
8                 return true;
9             } else if (mat[row][col] > target) {
10                 col--;    // current value too large: move left
11             } else {
12                 row++;    // current value too small: move down
13             }
14         }
15         return false;
16     }
17
18     public static void main(String[] args) {
19         int[][] mat = {
20             {1, 4, 7, 11},
21             {2, 5, 8, 12},
```

```

22         {3, 6, 9, 16},
23         {10, 13, 14, 17}
24     };
25     System.out.println(searchSorted(mat, 5)); // true
26     System.out.println(searchSorted(mat, 20)); // false
27 }
28 }

```

Listing 10.5: Search in Row-Column Sorted Matrix — $O(m+n)$

Searching for 5 in the matrix above (start top-right = mat[0

[3] = 11)]

row	col	mat[row][col]	Action
0	3	11	11 > 5: col-
0	2	7	7 > 5: col-
0	1	4	4 < 5: row++
1	1	5	5 = 5: Found! return true

Interview Corner

Why top-right corner? From top-right, moving left decreases value, moving down increases value — gives us a clear binary decision at every step. Starting from top-left or bottom-right would not give this property.

Complexity Analysis

Brute Force: $O(m \times n)$ Staircase (sorted matrix): $O(m + n)$ Space: $O(1)$ both

10.6 Check if Matrix is Symmetric

A matrix is **symmetric** if $\text{mat}[i][j] == \text{mat}[j][i]$ for all i, j . Only applicable to square matrices.

```

1 public class SymmetricMatrix {
2     public static boolean isSymmetric(int[][] mat) {
3         int n = mat.length;
4         // Only check upper triangle (above diagonal)
5         for (int i = 0; i < n; i++) {
6             for (int j = i + 1; j < n; j++) { // j starts from i+1
7                 if (mat[i][j] != mat[j][i]) {
8                     return false; // violation found
9                 }

```

```

10         }
11     }
12     return true;
13 }
14
15 public static void main(String[] args) {
16     int[][] sym = {
17         {1, 2, 3},
18         {2, 5, 6},
19         {3, 6, 9}
20     };
21     int[][] notSym = {
22         {1, 2, 3},
23         {4, 5, 6},
24         {7, 8, 9}
25     };
26     System.out.println(isSymmetric(sym));    // true
27     System.out.println(isSymmetric(notSym)); // false
28 }
29 }

```

Listing 10.6: Check Symmetric — only compare upper triangle, $O(n^2/2)$ **Check sym matrix: only upper triangle pairs**

i	j	mat[i][j]	mat[j][i]	Equal?
0	1	2	2	Yes
0	2	3	3	Yes
1	2	6	6	Yes
No violation found → Symmetric				

Why start j from i+1?

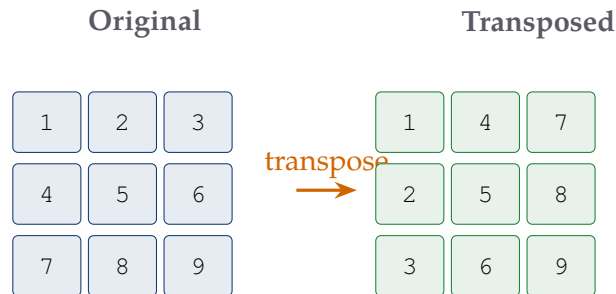
The diagonal (`mat[i][i]`) is always equal to itself. Below-diagonal pairs are mirror images of above-diagonal — checking both would be redundant. Starting j from $i + 1$ checks each pair exactly once: $O(n^2/2) \approx O(n^2)$.

Complexity Analysis

Time: $O(n^2)$ (half the pairs) Space: $O(1)$

10.7 Matrix Transpose

The **transpose** of a matrix flips it over its diagonal: row i becomes column i . $\text{mat}[i][j]$ and $\text{mat}[j][i]$ are swapped.



```

1 public class Transpose {
2     public static void transpose(int[][] mat) {
3         int n = mat.length;
4         // Only swap above-diagonal elements (j > i)
5         // If we swapped all, we'd undo every swap!
6         for (int i = 0; i < n; i++) {
7             for (int j = i + 1; j < n; j++) {
8                 int temp = mat[i][j];
9                 mat[i][j] = mat[j][i];
10                mat[j][i] = temp;
11            }
12        }
13    }
14
15    public static void main(String[] args) {
16        int[][] mat = {
17            {1, 2, 3},
18            {4, 5, 6},
19            {7, 8, 9}
20        };
21        transpose(mat);
22        // mat is now:
23        // 1 4 7
24        // 2 5 8
25        // 3 6 9
26        for (int[] row : mat) {
27            for (int v : row) System.out.print(v + " ");
28            System.out.println();
29        }
30    }
31 }

```

Listing 10.7: Matrix Transpose — in-place, swap upper triangle only

Transpose of $\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$

i	j	Swap	Effect
0	1	$\text{mat}[0][1] \leftrightarrow \text{mat}[1][0]$	$2 \leftrightarrow 4$
0	2	$\text{mat}[0][2] \leftrightarrow \text{mat}[2][0]$	$3 \leftrightarrow 7$
1	2	$\text{mat}[1][2] \leftrightarrow \text{mat}[2][1]$	$6 \leftrightarrow 8$

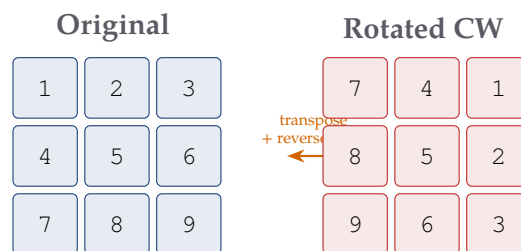
Result: $\{\{1,4,7\}, \{2,5,8\}, \{3,6,9\}\}$

Complexity Analysis

Time: $O(n^2)$ Space: $O(1)$ — in-place, only one temp variable

10.8 Rotate Matrix 90° Clockwise

Key insight: Rotate 90° clockwise = **Transpose** + **Reverse each row**.



```

1 public class RotateCW {
2     public static void rotate(int[][] mat) {
3         int n = mat.length;
4
5         // Step 1: Transpose the matrix
6         for (int i = 0; i < n; i++) {
7             for (int j = i + 1; j < n; j++) {
8                 int temp = mat[i][j];
9                 mat[i][j] = mat[j][i];
10                mat[j][i] = temp;
11            }
12        }
13
14        // Step 2: Reverse each row
15        for (int i = 0; i < n; i++) {
16            int left = 0, right = n - 1;
17            while (left < right) {
18                int temp = mat[i][left];
19                mat[i][left] = mat[i][right];

```

```

20         mat[i][right] = temp;
21         left++;
22         right--;
23     }
24 }
25 }
26
27 public static void main(String[] args) {
28     int[][] mat = {
29         {1, 2, 3},
30         {4, 5, 6},
31         {7, 8, 9}
32     };
33     rotate(mat);
34     for (int[] row : mat) {
35         for (int v : row) System.out.print(v + " ");
36         System.out.println();
37     }
38     // Output:
39     // 7 4 1
40     // 8 5 2
41     // 9 6 3
42 }
43 }

```

Listing 10.8: Rotate Matrix 90° Clockwise — Transpose + Reverse Rows

Rotate {{1,2,3},{4,5,6},{7,8,9}} clockwise

Step	Operation	Matrix State
Initial	—	{{1,2,3},{4,5,6},{7,8,9}}
After Transpose	swap upper triangle	{{1,4,7},{2,5,8},{3,6,9}}
After Rev Row 0	reverse {1,4,7}	{{7,4,1},{2,5,8},{3,6,9}}
After Rev Row 1	reverse {2,5,8}	{{7,4,1},{8,5,2},{3,6,9}}
After Rev Row 2	reverse {3,6,9}	{{7,4,1},{8,5,2},{9,6,3}}

Complexity Analysis

Time: $O(n^2)$ Space: $O(1)$ — fully in-place

10.9 Rotate Matrix 90° Anticlockwise

Key insight: Rotate 90° anticlockwise = **Transpose** + **Reverse each column**.

```
1 public class RotateCCW {
2     public static void rotateAnti(int[][] mat) {
3         int n = mat.length;
4
5         // Step 1: Transpose
6         for (int i = 0; i < n; i++) {
7             for (int j = i + 1; j < n; j++) {
8                 int temp = mat[i][j];
9                 mat[i][j] = mat[j][i];
10                mat[j][i] = temp;
11            }
12        }
13
14        // Step 2: Reverse each COLUMN (not row!)
15        for (int j = 0; j < n; j++) {
16            int top = 0, bottom = n - 1;
17            while (top < bottom) {
18                int temp = mat[top][j];
19                mat[top][j] = mat[bottom][j];
20                mat[bottom][j] = temp;
21                top++;
22                bottom--;
23            }
24        }
25    }
26
27    public static void main(String[] args) {
28        int[][] mat = {
29            {1, 2, 3},
30            {4, 5, 6},
31            {7, 8, 9}
32        };
33        rotateAnti(mat);
34        for (int[] row : mat) {
35            for (int v : row) System.out.print(v + " ");
36            System.out.println();
37        }
38        // Output:
39        // 3 6 9
40        // 2 5 8
41        // 1 4 7
42    }
43 }
```

Listing 10.9: Rotate Matrix 90° Anticlockwise — Transpose + Reverse Columns

Rotate $\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$ anticlockwise

Step	Operation	Matrix State
Initial	—	$\{\{1,2,3\},\{4,5,6\},\{7,8,9\}\}$
After Transpose	swap upper triangle	$\{\{1,4,7\},\{2,5,8\},\{3,6,9\}\}$
After Rev Col 0	reverse col $\{1,2,3\}$	$\{\{3,4,7\},\{2,5,8\},\{1,6,9\}\}$
After Rev Col 1	reverse col $\{4,5,6\}$	$\{\{3,6,7\},\{2,5,8\},\{1,4,9\}\}$
After Rev Col 2	reverse col $\{7,8,9\}$	$\{\{3,6,9\},\{2,5,8\},\{1,4,7\}\}$

CW vs CCW Memory Trick

Clockwise: Transpose → Reverse **rows**

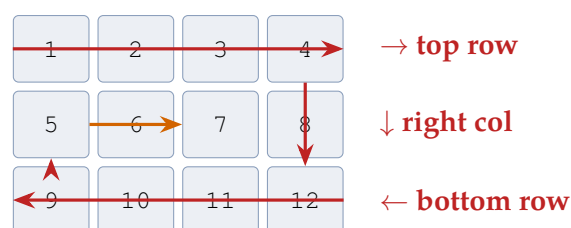
Anticlockwise: Transpose → Reverse **columns**

Complexity Analysis

Time: $O(n^2)$ Space: $O(1)$

10.10 Spiral Matrix — Print in Spiral Order

Problem: Print all elements of an $m \times n$ matrix in **clockwise spiral order**, starting from the top-left corner.



Approach: Maintain four boundaries: top, bottom, left, right. Traverse all four sides, then shrink boundaries inward. Repeat until boundaries cross.

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class SpiralMatrix {
5     public static List<Integer> spiralOrder(int[][] mat) {
6         List<Integer> result = new ArrayList<>();
7         if (mat == null || mat.length == 0) return result;
8
9         int top = 0;

```

```
10     int bottom = mat.length - 1;
11     int left    = 0;
12     int right   = mat[0].length - 1;
13
14     while (top <= bottom && left <= right) {
15
16         // 1. Traverse top row (left to right)
17         for (int j = left; j <= right; j++)
18             result.add(mat[top][j]);
19         top++;
20
21         // 2. Traverse right column (top to bottom)
22         for (int i = top; i <= bottom; i++)
23             result.add(mat[i][right]);
24         right--;
25
26         // 3. Traverse bottom row (right to left) -- if still valid
27         if (top <= bottom) {
28             for (int j = right; j >= left; j--)
29                 result.add(mat[bottom][j]);
30             bottom--;
31         }
32
33         // 4. Traverse left column (bottom to top) -- if still valid
34         if (left <= right) {
35             for (int i = bottom; i >= top; i--)
36                 result.add(mat[i][left]);
37             left++;
38         }
39     }
40     return result;
41 }
42
43 public static void main(String[] args) {
44     int[][] mat = {
45         {1, 2, 3, 4},
46         {5, 6, 7, 8},
47         {9, 10, 11, 12}
48     };
49     System.out.println(spiralOrder(mat));
50     // Output: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
51 }
52 }
```

Listing 10.10: Spiral Matrix — boundary shrinking approach

Spiral order of 3×4 matrix

Pass	Direction	Elements	Boundary after
1	Top row →	1, 2, 3, 4	top=1
2	Right col ↓	8, 12	right=2
3	Bottom row ←	11, 10, 9	bottom=1
4	Left col ↑	5	left=1
5	Top row →	6, 7	top=2 (done)

Output: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]

Complexity Analysis

Time: $O(m \times n)$ — every element visited exactly once Space: $O(1)$ extra (output list not counted)

Interview Corner

- The guard conditions `if (top <= bottom)` and `if (left <= right)` before steps 3 and 4 prevent double-counting rows/columns in non-square matrices
- Common follow-up: generate an $n \times n$ spiral matrix (fill values in spiral order)
- Appears in: Amazon, Google, Microsoft interviews

10.11 2-D Arrays — Quick Revision

Problem	Time	Key Idea
Print Diagonal	$O(n)$	<code>mat[i][i]</code>
Diagonal Sum	$O(n)$	Accumulate <code>mat[i][i]</code>
Search (brute)	$O(mn)$	Nested loop check all
Search (sorted)	$O(m + n)$	Top-right: left or down
Symmetric Check	$O(n^2)$	<code>mat[i][j] == mat[j][i]</code>
Transpose	$O(n^2)$	Swap <code>mat[i][j]</code> and <code>mat[j][i]</code>
Rotate CW 90°	$O(n^2)$	Transpose + reverse rows
Rotate CCW 90°	$O(n^2)$	Transpose + reverse columns
Spiral Order	$O(mn)$	4 boundaries: shrink inward
4-Sum	$O(n^3)$	Fix 2 + Two-Pointer on rest
Next Permutation	$O(n)$	Pivot + successor + reverse suffix

APNA CLG Quick Revision (from lecture notes)

- **Diagonal** → `mat[i][i]`
- **Search** → check all elements (brute), or top-right staircase (sorted)
- **Symmetric** → `mat[i][j] == mat[j][i]`
- **Transpose** → swap `mat[i][j] ↔ mat[j][i]`
- **Rotate CW** → Transpose + reverse rows
- **Rotate CCW** → Transpose + reverse columns

Array Problems — Hard Level

These problems require combining multiple techniques — sorting, divide and conquer, merge sort trees, or mathematical insight. All are directly asked at top-tier companies.

11.1 Merge Two Sorted Arrays Without Extra Space

Problem: Given two sorted arrays `arr1[]` (size m) and `arr2[]` (size n), merge them in-place so that `arr1` contains the m smallest elements and `arr2` contains the n largest, both sorted.

Approach — Gap Method (Shell Sort idea): Start with $\text{gap} = \lceil (m + n)/2 \rceil$. Compare elements that are `gap` apart across both arrays and swap if out of order. Halve the gap each time. Stop when $\text{gap} = 0$.

```

1 public class MergeWithoutSpace {
2
3     // Swap arr1[i] and arr2[j] if arr1[i] > arr2[j]
4     static void swapIfGreater(int[] arr1, int[] arr2, int i, int j) {
5         if (arr1[i] > arr2[j]) {
6             int temp = arr1[i];
7             arr1[i] = arr2[j];
8             arr2[j] = temp;
9         }
10    }
11
12    static void merge(int[] arr1, int[] arr2) {
13        int m = arr1.length;
14        int n = arr2.length;
15
16        // Initial gap: ceil of (m+n)/2
17        int gap = (m + n + 1) / 2;
18
19        while (gap > 0) {
20            int left = 0;
21            int right = left + gap;
22
23            while (right < m + n) {
24                // Case 1: both pointers in arr1
25                if (left < m && right < m) {

```

```

26         swapIfGreater(arr1, arr1, left, right);
27     }
28     // Case 2: left in arr1, right in arr2
29     else if (left < m && right >= m) {
30         swapIfGreater(arr1, arr2, left, right - m);
31     }
32     // Case 3: both pointers in arr2
33     else {
34         swapIfGreater(arr2, arr2, left - m, right - m);
35     }
36     left++;
37     right++;
38 }
39
40 // Halve the gap (ceil division)
41 if (gap == 1) break;
42 gap = (gap + 1) / 2;
43 }
44 }
45
46 public static void main(String[] args) {
47     int[] arr1 = {1, 3, 5, 7};
48     int[] arr2 = {0, 2, 6, 8, 9};
49     merge(arr1, arr2);
50     // arr1 = {0, 1, 2, 3}   arr2 = {5, 6, 7, 8, 9}
51     for (int x : arr1) System.out.print(x + " ");
52     System.out.println();
53     for (int x : arr2) System.out.print(x + " ");
54 }
55 }

```

Listing 11.1: Merge Without Extra Space — Gap Method, $O(n \log n)$ time $O(1)$ space

arr1 = {1,3,5,7}, arr2 = {0,2,6,8,9}, combined size = 9

Gap	Comparisons / Swaps	State after pass
5	Positions (0,5),(1,6),(2,7),(3,8)	arr1={0,2,5,7}, arr2={1,3,6,8,9}
3	Positions (0,3),(1,4)...(5,8)	arr1={0,1,2,3}, arr2={5,6,7,8,9}
2	No swaps needed	unchanged
1	No swaps needed	unchanged
Result: arr1={0,1,2,3} arr2={5,6,7,8,9}		

Complexity Analysis

Time: $O((m + n) \log(m + n))$ — gap halves each pass, each pass is $O(m + n)$ Space: $O(1)$ — fully in-place

Interview Corner

- Brute force: copy both to temp array, sort, copy back — $O((m + n) \log(m + n))$ but $O(m + n)$ space
- The gap method achieves same time complexity but $O(1)$ space — that is what interviewers want
- The gap calculation uses ceiling division: $\text{gap} = (\text{gap} + 1) / 2$
- Always handle 3 cases: both in arr1, split across arr1/arr2, both in arr2

11.2 Maximum Product Subarray

Problem: Find the contiguous subarray with the largest product.

Key insight: Unlike sum (Kadane's), negatives complicate products — two negatives make a positive. So track both **maximum** and **minimum** product ending at each index. When we see a negative number, the max and min swap roles.

```

1 public class MaxProductSubarray {
2
3     public static int maxProduct(int[] arr) {
4         int maxProd = arr[0]; // best product seen overall
5         int minP     = arr[0]; // min product ending here (tracks negative chain)
6         int maxP     = arr[0]; // max product ending here
7
8         for (int i = 1; i < arr.length; i++) {
9             // When arr[i] is negative, max and min swap roles
10            // So compute both and take the correct one
11            int tempMax = Math.max(arr[i], Math.max(maxP * arr[i], minP * arr[i]));
12            int tempMin = Math.min(arr[i], Math.min(maxP * arr[i], minP * arr[i]));
13
14            maxP = tempMax;
15            minP = tempMin;
16            maxProd = Math.max(maxProd, maxP);
17        }
18        return maxProd;
19    }
20
21    public static void main(String[] args) {
22        int[] arr1 = {2, 3, -2, 4};
23        System.out.println(maxProduct(arr1)); // 6 (subarray {2, 3})

```

```

24
25     int[] arr2 = {-2, 0, -1};
26     System.out.println(maxProduct(arr2)); // 0
27
28     int[] arr3 = {-2, 3, -4};
29     System.out.println(maxProduct(arr3)); // 24 (whole array)
30 }
31 }

```

Listing 11.2: Maximum Product Subarray — $O(n)$ time $O(1)$ space

arr = {2, 3, -2, 4}

i	arr[i]	maxP	minP	maxProd
init	2	2	2	2
1	3	6	3	6
2	-2	-2	-12	6
3	4	4	-48	6
Result: 6 (subarray {2, 3})				

Complexity AnalysisTime: $O(n)$ — single pass Space: $O(1)$ **Interview Corner**

- **Why track min?** A very negative product $\times$ a negative number = large positive. The min product can become the max at any point
- **What does zero do?** Resets both maxP and minP — subarray cannot span across a zero beneficially
- **Compare with Kadane's:** Kadane's tracks one variable (currentSum). Max product tracks two (maxP, minP) because multiplication is not monotone
- Alternative: prefix product $\times$ suffix product approach also works in $O(n)$

11.3 Find the Repeating and Missing Number

Problem: Given an array of size n containing integers from 1 to n where one number is repeated and one is missing, find both.

Approach — Math (Sum and Sum of Squares): Let repeating number = A , missing = B .

$$S_{\text{actual}} - S_{\text{expected}} = A - B \quad (\text{Equation 1})$$

$$S2_{\text{actual}} - S2_{\text{expected}} = A^2 - B^2 = (A - B)(A + B) \quad (\text{Equation 2})$$

Divide Eq.2 by Eq.1 to get $A + B$, then solve.

```
1 public class RepeatMissing {
2
3     public static int[] findRepeatMissing(int[] arr) {
4         int n = arr.length;
5
6         // Expected sums for 1..n
7         long Sn = (long) n * (n + 1) / 2;           // sum of 1..n
8         long S2n = (long) n * (n + 1) * (2 * n + 1) / 6; // sum of squares 1..n
9
10        // Actual sums
11        long S = 0, S2 = 0;
12        for (int x : arr) {
13            S += x;
14            S2 += (long) x * x;
15        }
16
17        // val1 = A - B    (from sum equation)
18        long val1 = S - Sn;
19        // val2 = A + B    (from sum-of-squares equation divided by val1)
20        long val2 = (S2 - S2n) / val1;
21
22        int A = (int) ((val1 + val2) / 2); // repeating
23        int B = (int) (val2 - A);         // missing
24
25        return new int[] {A, B};
26    }
27
28    public static void main(String[] args) {
29        int[] arr = {3, 1, 3, 4, 2}; // repeating=3, missing=5
30        int[] result = findRepeatMissing(arr);
31        System.out.println("Repeating: " + result[0]); // 3
32        System.out.println("Missing:    " + result[1]); // 5
33    }
34 }
```

Listing 11.3: Repeating and Missing — Math approach, $O(n)$ time $O(1)$ space

`arr = {3, 1, 3, 4, 2}, n = 5`

Calculation	Value
Expected sum $S_n = 5 \times 6/2$	15
Actual sum $S = 3 + 1 + 3 + 4 + 2$	13
$val1 = S - S_n = 13 - 15$	-2 (i.e. $A - B = -2$)
Expected $S2_n = 5 \times 6 \times 11/6$	55
Actual $S2 = 9 + 1 + 9 + 16 + 4$	39
$val2 = (39 - 55)/(-2)$	8 (i.e. $A + B = 8$)
$A = (-2 + 8)/2$	3 (repeating)
$B = 8 - 3$	5 (missing)

Complexity Analysis

Time: $O(n)$ Space: $O(1)$

Use `long` for sums — n^2 can overflow `int` for large n .

Interview Corner

- **Alternative: XOR approach** — XOR all elements with 1 to n ; the result is $A \oplus B$. Then find a set bit to separate the two groups. Also $O(n)$, $O(1)$ space, no overflow risk
- **Alternative: count array** — count occurrences in $O(n)$ time but $O(n)$ space
- The math approach is elegant but must use `long` to prevent overflow
- Always verify: plug A and B back to confirm $A - B = val1$

11.4 Majority Element II

Problem: Find all elements that appear more than $\lfloor n/3 \rfloor$ times. (There can be **at most 2** such elements.)

Approach — Extended Boyer-Moore Voting: Maintain two candidates and two counters. First pass finds candidates; second pass verifies them.

```

1 import java.util.ArrayList;
2 import java.util.List;
3
4 public class MajorityElementII {
5
6     public static List<Integer> majorityElement(int[] arr) {
7         // Phase 1: Find at most 2 candidates
8         int cand1 = 0, cand2 = 0;
```

```
9      int cnt1 = 0, cnt2 = 0;
10
11     for (int x : arr) {
12         if (x == cand1) {
13             cnt1++; // vote for candidate 1
14         } else if (x == cand2) {
15             cnt2++; // vote for candidate 2
16         } else if (cnt1 == 0) {
17             cand1 = x; cnt1 = 1; // new candidate 1
18         } else if (cnt2 == 0) {
19             cand2 = x; cnt2 = 1; // new candidate 2
20         } else {
21             cnt1--; cnt2--; // cancel one vote from each
22         }
23     }
24
25     // Phase 2: Verify both candidates actually exceed n/3
26     cnt1 = 0; cnt2 = 0;
27     for (int x : arr) {
28         if (x == cand1) cnt1++;
29         else if (x == cand2) cnt2++;
30     }
31
32     List<Integer> result = new ArrayList<>();
33     int n = arr.length;
34     if (cnt1 > n / 3) result.add(cand1);
35     if (cnt2 > n / 3) result.add(cand2);
36     return result;
37 }
38
39 public static void main(String[] args) {
40     int[] arr = {3, 2, 3, 1, 3, 2, 2};
41     System.out.println(majorityElement(arr)); // [3, 2]
42
43     int[] arr2 = {1};
44     System.out.println(majorityElement(arr2)); // [1]
45 }
46 }
```

Listing 11.4: Majority Element II — Extended Boyer-Moore, $O(n)$ time $O(1)$ space

`arr = {3, 2, 3, 1, 3, 2, 2}`, $n/3 = 2.33$ so threshold = 3

x	cand1	cnt1	cand2	cnt2	Action
3	3	1	0	0	new cand1
2	3	1	2	1	new cand2
3	3	2	2	1	vote cand1
1	3	1	2	0	cancel both
3	3	2	2	0	vote cand1
2	3	2	2	1	new cand2
2	3	2	2	2	vote cand2

Verify: 3 appears 3 times > 2.33 ✓ 2 appears 3 times > 2.33 ✓
Result: [3, 2]

Complexity Analysis

Time: $O(n)$ — two passes Space: $O(1)$

Interview Corner

- **Why at most 2 elements?** If 3 or more elements each exceeded $n/3$ occurrences, total count would exceed n — impossible
- **Difference from Majority Element I:** ME-I has one candidate (threshold $n/2$); ME-II has two candidates (threshold $n/3$). Generalises to $k - 1$ candidates for threshold n/k
- The verification phase is mandatory — Phase 1 only guarantees *candidates*, not winners
- **HashMap approach:** Count frequencies in $O(n)$ time + $O(n)$ space — valid but not optimal

11.5 Count Inversions

Problem: Count the number of pairs (i, j) where $i < j$ but `arr[i] > arr[j]`. An inversion is a pair that is “out of order”.

Approach — Modified Merge Sort: During the merge step, whenever an element from the right half is placed before elements from the left half, all remaining left-half elements form inversions with it. Count them.

```

1 public class CountInversions {
2     static long count = 0; // global inversion count
3 }

```

```

4     static void merge(int[] arr, int left, int mid, int right) {
5         int n1 = mid - left + 1;
6         int n2 = right - mid;
7         int[] L = new int[n1];
8         int[] R = new int[n2];
9
10        for (int i = 0; i < n1; i++) L[i] = arr[left + i];
11        for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
12
13        int i = 0, j = 0, k = left;
14        while (i < n1 && j < n2) {
15            if (L[i] <= R[j]) {
16                arr[k++] = L[i++]; // no inversion
17            } else {
18                // L[i] > R[j]: all remaining left elements form inversions
19                // with R[j]
20                count += (n1 - i); // this is the key step
21                arr[k++] = R[j++];
22            }
23        }
24        while (i < n1) arr[k++] = L[i++];
25        while (j < n2) arr[k++] = R[j++];
26    }
27
28    static void mergeSort(int[] arr, int left, int right) {
29        if (left < right) {
30            int mid = (left + right) / 2;
31            mergeSort(arr, left, mid);
32            mergeSort(arr, mid + 1, right);
33            merge(arr, left, mid, right);
34        }
35    }
36
37    public static void main(String[] args) {
38        int[] arr = {5, 4, 3, 2, 1};
39        count = 0;
40        mergeSort(arr, 0, arr.length - 1);
41        System.out.println("Inversions: " + count); // 10
42    }

```

Listing 11.5: Count Inversions — Modified Merge Sort, $O(n \log n)$ time $O(n)$ space

arr = {3, 2, 1}

Merge step	Comparison	Inversions added
Merge {3} and {2}	3 > 2: all 1 left element inverts with 2	count = 1
Merge {2,3} and {1}	2 > 1: 2 remaining left elements invert	count = 3
Total inversions: 3 (pairs: (3,2), (3,1), (2,1))		

Complexity Analysis

Time: $O(n \log n)$ — same as merge sort Space: $O(n)$ for temp arrays

Brute force ($O(n^2)$): check all pairs (i, j) with $i < j$ and $\text{arr}[i] > \text{arr}[j]$.

Interview Corner

- **Why does merge sort work?** During merge, the left and right halves are already sorted. If $L[i] > R[j]$, then all elements from $L[i]$ to end of L are greater than $R[j]$ — hence $n1 - i$ inversions in one step
- Use `long` for the count — fully reversed array of size n has $n(n - 1)/2$ inversions, which overflows `int` for large n
- Inversions measure “sortedness” — 0 inversions = sorted, $n(n - 1)/2$ = reverse sorted
- Asked at Amazon, Microsoft, Adobe

11.6 Reverse Pairs

Problem: Count the number of *important reverse pairs*: pairs (i, j) where $i < j$ and $\text{arr}[i] > 2 * \text{arr}[j]$.

Approach — Modified Merge Sort: Similar to Count Inversions but the condition is $\text{arr}[i] > 2 * \text{arr}[j]$ (not just $\text{arr}[i] > \text{arr}[j]$). Count pairs *before* merging (using two pointers on sorted halves), then merge normally.

```

1 public class ReversePairs {
2     static long count = 0;
3
4     static void countPairs(int[] arr, int left, int mid, int right) {
5         // Both halves are sorted at this point
6         // For each i in left half, find how many j in right half satisfy arr[i]
7         // > 2*arr[j]
8         int j = mid + 1;
9         for (int i = left; i <= mid; i++) {
10            // Advance j while condition holds (right half is sorted so j only
11            // moves right)
12            while (j <= right && arr[i] > 2L * arr[j]) {
13                j++;
14            }
15            count += (j - (mid + 1)); // all elements from mid+1 to j-1 form
16            // pairs with arr[i]
17        }
18    }
19
20    static void merge(int[] arr, int left, int mid, int right) {
21        int n1 = mid - left + 1;
22        int n2 = right - mid;
23        int[] L = new int[n1];

```

```

21     int[] R = new int[n2];
22
23     for (int i = 0; i < n1; i++) L[i] = arr[left + i];
24     for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];
25
26     int i = 0, k = left;
27     int jj = 0;
28     while (i < n1 && jj < n2) {
29         if (L[i] <= R[jj]) arr[k++] = L[i++];
30         else arr[k++] = R[jj++];
31     }
32     while (i < n1) arr[k++] = L[i++];
33     while (jj < n2) arr[k++] = R[jj++];
34 }
35
36 static void mergeSort(int[] arr, int left, int right) {
37     if (left < right) {
38         int mid = (left + right) / 2;
39         mergeSort(arr, left, mid);
40         mergeSort(arr, mid + 1, right);
41         countPairs(arr, left, mid, right); // count BEFORE merging
42         merge(arr, left, mid, right);
43     }
44 }
45
46 public static void main(String[] args) {
47     int[] arr = {1, 3, 2, 3, 1};
48     count = 0;
49     mergeSort(arr, 0, arr.length - 1);
50     System.out.println("Reverse pairs: " + count); // 2
51     // Pairs: (3,1) at indices (1,4) and (3,1) at indices (3,4)
52 }
53 }

```

Listing 11.6: Reverse Pairs — Modified Merge Sort, $O(n \log n)$

arr = {1, 3, 2, 3, 1}

Count pairs phase (left sorted, right sorted)	Pairs found
Left={1,3}, Right={2,3,1}	$3 > 2 \times 1 = 2$: 1 pair (3,1)
Left={1,2,3,3}, Right={1}	$3 > 2, 3 > 2$: 2 pairs (3,1),(3,1)
Total: 2 reverse pairs	

Complexity Analysis

Time: $O(n \log n)$ — merge sort with an extra linear pass per level Space: $O(n)$

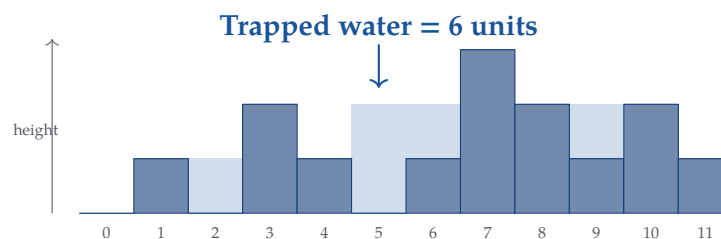
Interview Corner

- **Key difference from Count Inversions:** The condition is $\text{arr}[i] > 2 * \text{arr}[j]$, not $\text{arr}[i] > \text{arr}[j]$. This means you *cannot* count pairs inside the merge step — you must count *before* merging, then merge separately
- Use $2L * \text{arr}[j]$ (long multiplication) to avoid overflow when $\text{arr}[j]$ is large
- The two-pointer in `countPairs` is valid because both halves are sorted: as i increases, j never needs to go back
- LeetCode 493 — asked at Google, Amazon

11.7 Trapping Rainwater

Problem: Given an array `height[]` where each element represents the height of a bar, find how much water can be trapped between the bars after it rains.

Example: `height = [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1]` → answer = 6 units



Key Insight: Water at index $i = \min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$.

The water level at any position is bounded by the *shorter* of the tallest bars on its left and right. If that minimum exceeds the current bar height, water accumulates.

11.7.1 Brute Force — $O(n^2)$

For each index, scan left and right to find the tallest bar on each side.

```

1 public class TrappingRainwaterBrute {
2     public static int trap(int[] height) {
3         int n = height.length;
4         int totalWater = 0;
5
6         for (int i = 1; i < n - 1; i++) {
7             // Find max height to the LEFT of i (including i)
8             int maxLeft = 0;
9             for (int j = 0; j <= i; j++) {
10                 maxLeft = Math.max(maxLeft, height[j]);
11             }
12
13             // Find max height to the RIGHT of i (including i)
14             int maxRight = 0;

```



```

15         for (int j = i; j < n; j++) {
16             maxRight = Math.max(maxRight, height[j]);
17         }
18
19         // Water above this bar = min boundary - bar height
20         totalWater += Math.min(maxLeft, maxRight) - height[i];
21     }
22     return totalWater;
23 }
24 }

```

Listing 11.7: Trapping Rainwater — Brute Force $O(n^2)$

Complexity Analysis

Time: $O(n^2)$ Space: $O(1)$

11.7.2 Prefix Array Approach — $O(n)$ time $O(n)$ space

Pre-compute `leftMax[]` and `rightMax[]` arrays, then calculate water in a single pass.

```

1  public class TrappingRainwaterPrefix {
2      public static int trap(int[] height) {
3          int n = height.length;
4          if (n == 0) return 0;
5
6          // leftMax[i] = max height from index 0 to i (inclusive)
7          int[] leftMax = new int[n];
8          // rightMax[i] = max height from index i to n-1 (inclusive)
9          int[] rightMax = new int[n];
10
11         leftMax[0] = height[0];
12         for (int i = 1; i < n; i++) {
13             leftMax[i] = Math.max(leftMax[i - 1], height[i]);
14         }
15
16         rightMax[n - 1] = height[n - 1];
17         for (int i = n - 2; i >= 0; i--) {
18             rightMax[i] = Math.max(rightMax[i + 1], height[i]);
19         }
20
21         // Water at each index = min(leftMax, rightMax) - height[i]
22         int totalWater = 0;
23         for (int i = 0; i < n; i++) {
24             totalWater += Math.min(leftMax[i], rightMax[i]) - height[i];
25         }
26         return totalWater;
27     }
28
29     public static void main(String[] args) {
30         int[] h = {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1};
31         System.out.println(trap(h)); // 6

```

```

32     }
33 }

```

Listing 11.8: Trapping Rainwater — Prefix Arrays $O(n)$

height = {0,1,0,2,1,0,1,3,2,1,2,1}

i	h[i]	leftMax	rightMax	min(L,R)	water
0	0	0	3	0	0
1	1	1	3	1	0
2	0	1	3	1	1
3	2	2	3	2	0
4	1	2	3	2	1
5	0	2	3	2	2
6	1	2	3	2	1
7	3	3	3	3	0
8	2	3	2	2	0
9	1	3	2	2	1
10	2	3	2	2	0
11	1	3	1	1	0
Total					6

Complexity Analysis

Time: $O(n)$ — three linear passes Space: $O(n)$ for two prefix arrays

11.7.3 Two Pointer Approach — $O(n)$ time $O(1)$ space (Optimal)

Use two pointers moving inward. At any point, the side with the smaller max determines the water level — no need to know the other side's max.

```

1 public class TrappingRainwaterTwoPointer {
2     public static int trap(int[] height) {
3         int left = 0, right = height.length - 1;
4         int leftMax = 0, rightMax = 0;
5         int totalWater = 0;
6
7         while (left < right) {
8             if (height[left] <= height[right]) {
9                 // Left side is the bottleneck
10                if (height[left] >= leftMax) {
11                    leftMax = height[left]; // update left max
12                } else {

```

```

13         totalWater += leftMax - height[left]; // water trapped
14     }
15     left++;
16 } else {
17     // Right side is the bottleneck
18     if (height[right] >= rightMax) {
19         rightMax = height[right]; // update right max
20     } else {
21         totalWater += rightMax - height[right]; // water trapped
22     }
23     right--;
24 }
25 }
26 return totalWater;
27 }
28
29 public static void main(String[] args) {
30     int[] h1 = {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1};
31     System.out.println(trap(h1)); // 6
32
33     int[] h2 = {4, 2, 0, 3, 2, 5};
34     System.out.println(trap(h2)); // 9
35 }
36 }

```

Listing 11.9: Trapping Rainwater — Two Pointer $O(n)$ time $O(1)$ space

height = {4, 2, 0, 3, 2, 5}

L	R	h[L]	h[R]	lMax	rMax	water	Action
0	5	4	5	0	0	0	lMax=4, L++
1	5	2	5	4	0	0	water+=2, L++
2	5	0	5	4	0	2	water+=4, L++
3	5	3	5	4	0	6	water+=1, L++
4	5	2	5	4	0	7	water+=2, L++
5	5	–	–	–	–	9	L↯R: stop

Result: 9

Complexity Analysis

Approach	Time	Space
Brute Force	$O(n^2)$	$O(1)$
Prefix Arrays	$O(n)$	$O(n)$
Two Pointer (optimal)	$O(n)$	$O(1)$

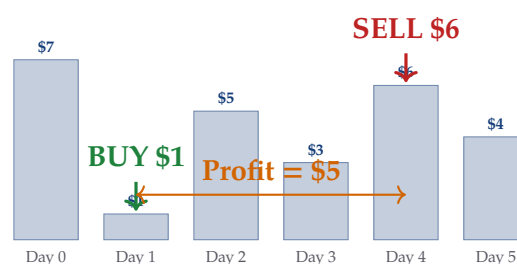
Interview Corner

- **Why two pointers work:** When $\text{height}[\text{left}] \leq \text{height}[\text{right}]$, the water at left is determined by leftMax alone — we already know $\text{rightMax} \geq \text{height}[\text{right}] \geq \text{height}[\text{left}]$, so the right boundary is never the bottleneck
- **Key formula:** water at index $i = \max(0, \min(\text{leftMax}, \text{rightMax}) - \text{height}[i])$
- LeetCode 42 — asked at Amazon, Google, Microsoft, Adobe
- Edge cases: empty array, single bar, monotonically increasing/decreasing (no water)

11.8 Best Time to Buy and Sell Stock

Problem: Given an array `prices[]` where `prices[i]` is the price of a stock on day i , find the maximum profit you can achieve. You may complete at most **one** transaction (buy one, sell one). You must buy before you sell.

Example: `prices = [7, 1, 5, 3, 6, 4]` → `answer = 5` (buy day 1 at \$1, sell day 4 at \$6)



Key Insight: Track the **minimum price seen so far** as you scan left to right. At each day, the best profit is `price[i] - minSoFar`. Keep the global maximum of all such profits.

```

1 public class BestTimeToBuySellStock {
2     public static int maxProfit(int[] prices) {
3         int minPrice = Integer.MAX_VALUE; // cheapest buying opportunity so far
4         int maxProfit = 0; // best profit seen so far
5
6         for (int i = 0; i < prices.length; i++) {
7

```

```

8         if (prices[i] < minPrice) {
9             minPrice = prices[i];           // found a cheaper day to buy
10        } else if (prices[i] - minPrice > maxProfit) {
11            maxProfit = prices[i] - minPrice; // found a better profit
12        }
13    }
14    return maxProfit;
15 }

16
17 public static void main(String[] args) {
18     int[] prices1 = {7, 1, 5, 3, 6, 4};
19     System.out.println(maxProfit(prices1)); // 5 (buy@1 sell@6)
20
21     int[] prices2 = {7, 6, 4, 3, 1};
22     System.out.println(maxProfit(prices2)); // 0 (prices always fall, no
23         profit)
24
25     int[] prices3 = {1, 2};
26     System.out.println(maxProfit(prices3)); // 1
27 }

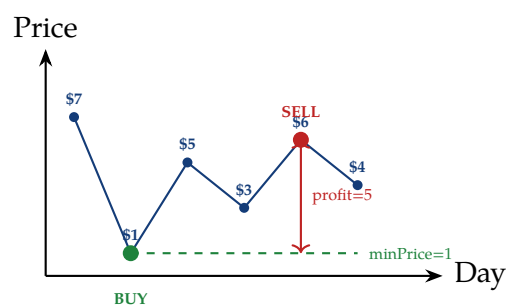
```

Listing 11.10: Best Time to Buy and Sell Stock — $O(n)$ time $O(1)$ space

prices = {7, 1, 5, 3, 6, 4}

i	prices[i]	minPrice	maxProfit	Note
0	7	7	0	New min: 7
1	1	1	0	New min: 1
2	5	1	4	Profit = 5-1 = 4
3	3	1	4	Profit = 3-1 = 2, not better
4	6	1	5	Profit = 6-1 = 5, new best!
5	4	1	5	Profit = 4-1 = 3, not better

Result: maxProfit = 5



Complexity Analysis

Time: $O(n)$ — single pass Space: $O(1)$ — only two variables

Interview Corner

- **Why single pass works:** We always want to buy at the *lowest price seen so far* and sell at the *highest price after that buy*. Scanning left to right, `minPrice` gives the best buy opportunity up to the current day
- **What if prices always fall?** `maxProfit` stays 0 — return 0, no transaction made
- **LeetCode 121** — one of the most-asked problems at every major company
- **Follow-up: Multiple transactions (LeetCode 122)** — you can buy and sell as many times as you want. Approach: greedily add every positive consecutive difference:
`if (prices[i] > prices[i-1]) profit += prices[i] - prices[i-1]`
- **Follow-up: At most 2 transactions (LeetCode 123)** — Dynamic programming with states
- Always check: is the sell day *after* the buy day? (The single-pass approach guarantees this since `minPrice` is always from a previous or current day)